

IMPROVING MICROSERVICES OBSERVABILITY IN CLOUD-NATIVE INFRASTRUCTURE USING EBPF

by

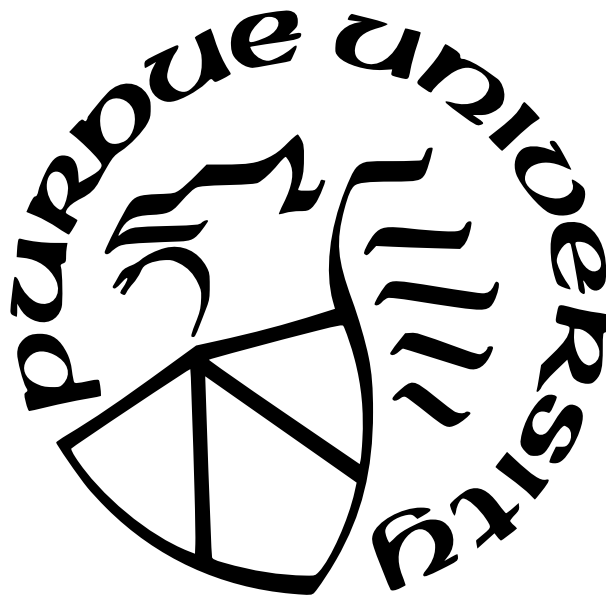
Bhavye Sharma

A Thesis

Submitted to the Faculty of Purdue University

In Partial Fulfillment of the Requirements for the degree of

Master of Science



Department of Computer and Information Technology

West Lafayette, Indiana

May 2023

**THE PURDUE UNIVERSITY GRADUATE SCHOOL
STATEMENT OF COMMITTEE APPROVAL**

Dr. Deepak Nadig, Chair

Department of Computer and Information Technology

Dr. Baijian Yang

Department of Computer and Information Technology

Dr. Abdul Salam

Department of Computer and Information Technology

Approved by:

Dr. John A. Springer

Dedicated to my mother.

ACKNOWLEDGMENTS

I gratefully acknowledge my thesis committee for their insightful comments and guidance and my family for their support and encouragement.

I would like to thank Professor Deepak Nadig for providing me with help, advice, support and all the resources I needed to successfully complete my thesis. Professor Deepak Nadig provided me with guidance and resources in conducting research and an environment of comfort and familiarity during my time in West Lafayette for which I am deeply appreciative.

I am also fortunate to have worked with my colleagues at Purdues CIT department and the Cloud-native, Cyberinfrastructure, and Networks Laboratory: Meher Hasanth, Ayush Shridhar, Isham Mahajan, Daniel Spencer, Akash Ravi, Avigyan Mukherjee and Kanwardeep Singh Walia. Discussing research with them is always enlightening. They also offered me lots of support.

I would like to thank Curtis Rissi, Young Jeong, Andrew Park, and Yaniv Bossem from AWS for helping me identify my problem by allowing me to interview them and seek feedback regarding the research topic “Observability For The Cloud”.

Lastly, I would like to thank Professor Nicole Key of the Purdue University Mechanical Engineering Department for hiring me as a Graduate Assistant software developer and helping me hone my professional skills as I start my career.

TABLE OF CONTENTS

LIST OF TABLES	8
LIST OF FIGURES	9
ABBREVIATIONS	11
ABSTRACT	12
1 INTRODUCTION	14
1.1 Objective	16
1.2 Motivation	16
1.3 Problem Statement	16
1.4 Significance of the Proposed Work	17
1.5 Research Questions	18
1.6 Contributions	19
1.7 Assumptions	19
1.8 Limitations of the Work	20
1.9 Delimitations	20
1.10 Organization of this Thesis	21
2 BACKGROUND AND RELATED WORK	22
2.1 Virtualization	22
2.1.1 Containers	22
2.1.2 Container Orchestration Systems	23
2.1.3 Microservices	24
2.2 Network Service Mesh	26
2.2.1 Data Plane	27
2.2.2 Control Plane	27
2.2.3 Ingress	28
2.3 Observability	28

2.3.1	Instrumentation for Observability	29
2.3.2	Opentelemetry	29
2.3.2.1	Collector	30
2.3.2.2	Agent	31
2.3.3	Prometheus	31
2.3.3.1	Node Exporter Agent	32
2.3.3.2	cAdvisor Agent	35
2.3.4	Extended Berkeley Packet Filter (eBPF)	37
2.3.4.1	Architecture	37
2.3.4.2	BPF Programming	39
2.3.4.3	BPF Maps	40
2.3.4.4	libbpf	42
2.3.4.5	BCC	42
2.3.4.6	Bpfttrace	43
2.4	Chaos Engineering	44
2.4.1	Chaos Mesh	45
2.5	Related Works	48
3	PROPOSED SOLUTION	50
3.1	System Design	50
3.2	Data Model	51
3.3	Application Layer Observability	52
3.4	Network Observability Agents	54
3.5	Node Observability Agents	55
3.6	Prometheus Collector	57
3.7	Evaluation Framework	58
4	EXPERIMENTAL RESULTS	61
4.1	Experiment Testbed	61
4.1.1	Baremetal Resource Reservation	61
4.1.2	Launching Instances	64

4.1.3	Configuring Kubernetes with cloud-init	64
4.2	Cloud-observability Framework	67
4.2.1	Prometheus Collector	67
4.2.2	Grafana Dashboard	68
4.2.3	Agents	68
4.2.4	Deployments for Evaluation	69
4.2.5	Chaos Mesh	71
4.3	Evaluation	71
4.3.1	Performance Benchmarks	72
4.3.2	Synthetic Load Generation	73
4.3.3	Stress Tests	76
4.3.4	I/O Performance	78
4.3.5	Network Performance	80
5	CONCLUSIONS AND FUTURE WORK	84
5.1	Conclusions	84
5.2	Future Research Directions	85
	REFERENCES	87

LIST OF TABLES

2.1	Bpftrace tools for kernel monitoring [58]	43
3.1	Microservices in our test application	53
3.2	eBPF scripts implemented for real-time kernel observability	57
4.1	CPU and RAM usage comparison	72
4.2	CPU and RAM observability	78

LIST OF FIGURES

1.1	Three pillars of observability i.e., metrics, logs and traces [9]	15
2.1	Sidecar proxy communication in service mesh networks [37]	26
2.2	OpenTelemetry architecture	30
2.3	Node exporter dashboard [45]	34
2.4	cAdvisor dashboard [47]	36
2.5	eBPF architecture	38
2.6	Example eBPF program	40
2.7	eBPF map types	41
2.8	eBPF map definition	41
2.9	Chaos Mesh dashboard [60]	47
3.1	Proposed system design	50
3.2	OTLP trace example	51
3.3	Span-trace data model [30]	52
3.4	Types of network traffic in the cloud [69]	54
3.5	Proposed solution agent-collector architecture	56
3.6	Locust load testing	60
4.1	Requesting lease on the chameleon cloud	62
4.2	Storage network for NFS on the Chameleon cloud	62
4.3	Launch a baremetal instance on Chameleon cloud	63
4.4	Using a cloud-init script to configure a baremetal Kubernetes cluster	64
4.5	cloud-init script for the control plane	66
4.6	Kubernetes YAML manifest for monitoring namespace	67
4.7	Deploying eBPF exporter	69
4.8	Deploying OpenTelemetry microservice demo	70
4.9	OpenTelemetry microservices demo architecture	70
4.10	CPU and RAM usage comparison (normalized)	73
4.11	Locust script for synthetic load generation	75
4.12	CPU and RAM stress test YAML configuration	76

4.13 Node exporter identifying high CPU and RAM load during stress test	77
4.14 cAdvisor identifying high resource consumption pod in cluster	77
4.15 I/O stress test	79
4.16 Block I/O latency histogram from eBPF exporter	79
4.17 OpenTelemetry distributed trace visualized on Grafana dashboard	80
4.18 cAdvisor and node exporter network monitoring	82

ABBREVIATIONS

AWS	Amazon Web Services
BCC	BPF Compiler Collection
BPF	Berkeley Packet Filter
CNI	Container Network Interface
eBPF	Extended Berkeley Packet Filter
HPC	High Performance computing
HPA	Horizontal Pod Auto-scaling
JIT	Just-in-time
JVM	Java Virtual Machine
K8s	Kubernetes
LLVM	Low-Level Virtual Machine (Compiler)
NFV	Network Function Virtualization
OTLP	Opentelemetry Protocol
REST	Representational State Transfer
RCA	Root Cause Analysis
RPC	Remote Procedure Call
SDN	Software Defined Network
SDK	Software Development Kit
SSH	Secure Shell
TCP	Transmission Control Protocol
USDT	User Statically-Defined Tracing
VM	Virtual Machine
OS	Operating System
OTEL	Opentelemetry
XDP	Express Data Path

ABSTRACT

Microservices have emerged as a popular pattern for developing large-scale applications in cloud environments for their flexibility, scalability, and agility benefits. However, microservices make management more complex due to their scale, multiple languages, and distributed nature. Orchestration and automation tools like Kubernetes help deploy microservices running simultaneously, but it can be difficult for an operator to understand their behaviors, interdependencies, and interactions. In such a complex and dynamic environment, performance problems (e.g., slow application responses and high resource usage) require significant human effort spent on diagnosis and recovery. Moreover, manual diagnosis of cloud microservices tends to be tedious, time-consuming, and impractical. Effective and automated performance analysis and anomaly detection require an observable system, which means an application’s internal state can be inferred by observing and tracking metrics, traces, and logs. Traditional APM uses libraries and SDKs to improve application monitoring and tracing but has additional overheads of rewriting, recompiling, and redeploying the applications’ code base. Therefore, there is a critical need for a standardized automated microservices observability solution that does not require rewriting or redeploying the application to keep up with the agility of microservices.

This thesis studies observability for microservices and implements an automated Extended Berkeley Packet Filter (eBPF) based observability solution. eBPF is a Linux feature that allows us to write extensions to the Linux kernel for security and observability use cases. eBPF does not require modifying the application layer and instrumenting the individual microservices. Instead, it instruments the kernel-level API calls, which are common across all hosts in the cluster. eBPF programs provide observability information from the lowest-level system calls and can export data without additional performance overhead. The Prometheus time-series database is leveraged to store all the captured metrics and traces for analysis. With the help of our tool, a DevOps engineer can easily identify abnormal behavior of microservices and enforce appropriate countermeasures. Using Chaos Mesh, we inject anomalies at the network and host layer, which we can identify with root cause identification

using the proposed solution. The Chameleon cloud testbed is used to deploy our solution and test its capabilities and limitations.

1. INTRODUCTION

Microservices are an architectural and organizational approach to software development where software is composed of small independent services that communicate over well-defined APIs. Most of the recent client-server web applications are adopting microservice architectures due to their benefits like modularity and scalability [1] over their monolith counterparts [2]. The highly networked microservice architecture brings in challenges of observability into the underlying functioning of the application, introduces networking overheads [3], and influences the architectural design of our applications [4]. To manage the growing complexity of microservices the concept of microservice mesh Linkerd [5] was introduced to abstract the service-to-service communication away from the application pod to a “sidecar” proxy. A service mesh abstracts networking logic into a parallel layer of infrastructure i.e. the data plane and the control plane. The control plane is responsible for configuring the network policies and routing paths inside our mesh whereas the data plane sidecar is responsible for using lightweight communication protocols like REST or RPC to communicate between the various microservices using the specified routes. Although the control plane can configure the microservices, configuration drift [6] is common in cloud environments which occurs when the actual known state of the infrastructure differs from the prior configuration. Therefore, there is a critical need for real-time observability in our service mesh to be able to identify performance issues and analyze the causes of latency.

The definition of observability for a software system is its “*capability to allow a human to ask and answer questions*” [7]. The more questions we can ask and answer about the system, the more observable it is. The open-source community defines observability with its three pillars i.e., metrics, logs and traces [8].

- **Metrics:** Metrics are numerical measurements that are obtained by observing a system from the outside. They represent data that is measured over specific intervals of time, and their power lies in the ability to use mathematical modeling and prediction to gain insight into the behavior of a system in the present and future. Due to their numeric nature, metrics can be easily stored, processed, compressed, and retrieved, making

them ideal for extended data retention and quick querying. As a result, metrics are well-suited for creating dashboards that illustrate historical trends.

- **Logs:** The sequence of events generated inside a singular application. Logs are records of discrete events that have occurred over time, and they are immutable and timestamped. They are particularly useful in revealing emergent and unpredictable behaviors that may be exhibited by components of a distributed system.
- **Traces:** The interaction amongst multiple microservices and the ability to obtain a macro view of the request-response lifecycle. Understanding the path of a request enables us to identify the various services involved in its execution, while analyzing the structure of the request provides a means to comprehend the impact of asynchrony at different stages of its execution.

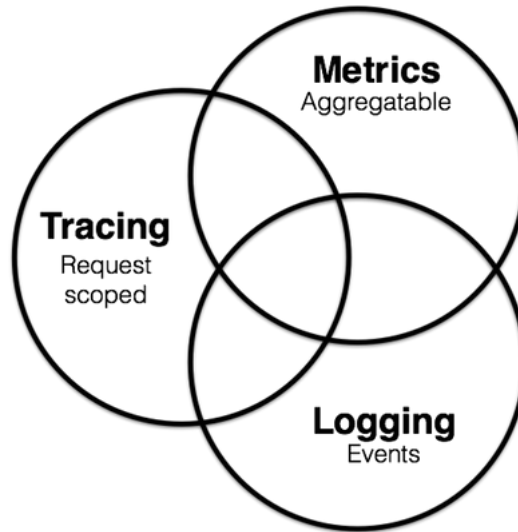


Figure 1.1. Three pillars of observability i.e., metrics, logs and traces [9]

In this work, we implement a service mesh observability and tracing framework to take application, network, host and kernel activity of containerised pods into consideration. In the following sections we present objective and problem statement of this work is described in detail.

1.1 Objective

The main objective of this work is to propose an observability solution framework for Kubernetes, implement, and evaluate the use cases. The goal of the framework is to collect metrics, logs and traces from every layer of our virtualized environment. This should allow DevOps engineers to identify bottlenecks in microservice applications and take countermeasures to fix them. The framework must be able to analyze all application-layer, network-layer and kernel-layer observability data generated. The framework should not add additional latency to the applications themselves and data must be measured accurately. Finally, the solution should require minimal rewriting of application codebase of the pods and plug-in to any Kubernetes deployment.

1.2 Motivation

With the growth of microservice development patterns in the cloud, it is becoming challenging to uniformly observe and monitor all the applications in a standardized format. The traditional approach of using language-specific libraries and software development kits (SDKs) requires rewriting logging functionality for each programming language microservice we deploy, sometimes including vendor lock-ins. Additionally, the shift to public cloud service providers (AWS, Azure, GCP) has reduced the ability of tenants to monitor the performance or behavior of their applications. In the on-premise server model, the operator can physically and virtually access its hardware to get in-depth, real-time information. This thesis aims to develop a Linux observability solution that allows us to get standardized metrics, logs, and traces directly from any application deployed in any public cloud provider as a containerized service without instrumentation. Our solution is compatible with any Linux kernel version greater than 3.18 and does not require rewriting or redeploying any application-layer code.

1.3 Problem Statement

The application-layer observability approach cannot keep up with the agility of the microservice software development approach since they require integrating additional code

within every application individually. These solutions cannot measure kernel-level performance metrics, such as CPU time, process waiting time, I/O time, and unauthorized system calls. Moreover, these libraries cannot fully take advantage of high-level virtualization, i.e., containers, and increase the memory and CPU footprint of the container images. Using eBPF, an in-kernel virtual execution engine, we explore the possibility of achieving in-depth observability for our microservices without modifying the application codebase.

It is important to note that companies like New Relic, AppDynamic, Sysdig, DynaTrace and GroundCover provide similar out-of-box solutions to monitor Kubernetes clusters using eBPF observability. However, all of them are either commercial solutions (New Relic, AppDynamic, DynaTrace), require the use of their Kubernetes networking CNI and other custom resources (Sysdig) or store cluster metadata information in their private cloud (GroundCover).

1.4 Significance of the Proposed Work

Large collections of small servers are a particularly cost-efficient platform for distributed services workloads [10]. With the widespread adoption of cloud computing for businesses, the microservice pattern has emerged as the de facto standard for application development. A clear majority (61%) of 1500 respondents to a study conducted [11] say their organizations have used microservices for between 1 and 5 years. In addition, 92% of organizations report experiencing success with microservices. Although promising, the same study also highlighted that 60% of respondents felt their organization lacked monitoring tools, and 40% of respondents felt the same regarding observability tools. During an interview we conducted on September 11, 2022, Curtis Rissi, Principal Architect, AWS, stated that there is a huge gap in the industry for cloud-native monitoring and tracing tools, especially for distributed and containerized applications.

Containers have revolutionized the management and utilization of cloud resources. Rather than providing each app an independent operating system like VMs, containers use namespaces to achieve isolation in a multi-tenant cloud environment. The benefit of this approach is the less main memory footprint and ease of sharing resources. The drawback of containers

is their ephemeral nature; they can be spun up and terminated immediately. This makes tracking container behavior challenging, as the process ID for every container instance can vary rapidly. Using application-layer instrumentation libraries like OpenTelemetry allows us to collect overall application metrics and logs, but they do not provide the pod-layer visibility of the application replicas. Kubernetes [12], the orchestrator behind most container deployments, keeps track of deployed containers and provides them with a unique identifier or IP address. The idea of using Kubernetes as a database for service discovery was proposed by [5], an open-source service mesh implementation. This approach has since gained more traction with other service mesh implementations like Cilium, Istio and Envoy using the same approach [13]. Although these service mesh implementations can achieve the same visibility and observability into our system’s network, they impose additional operational costs of maintaining a sidecar proxy for each microservice. The sidecar also introduces costs and latency to our distributed application due to additional networking calls. By replacing the sidecar-based observability with in-kernel eBPF functions, we believe we can drastically improve performance and visibility and reduce overheads.

1.5 Research Questions

Our work endeavors to answer the following research questions:

- RQ1 What are the capabilities and limitations of non-eBPF-based observability tools for Kubernetes clusters? (e.g., Node Exporter, cAdvisor, OpenTelemetry)
- RQ2 Will using the Extended Berkeley Packet Filter (eBPF) for Kubernetes observability improve the accuracy and granularity of the collected metrics compared to application layer standards of OpenTelemetry?
- RQ3 Can we perform real-time anomaly detection using our proposed multi-layer observability approach?

1.6 Contributions

This thesis presents an automated multi-layered observability solution with an open-sourced implementation on the Linux platform for determining and instrumenting the critical metrics, logs and traces in distributed Kubernetes workloads. We collect data from distributed microservice-based workloads and visualize it on Grafana dashboards. We utilize application workloads provided by OpenTelemetry demo and deploy it on our Kubernetes cluster. Our solution can extract detailed observability metrics from the Kubernetes pods without modifying the application’s source code. By attaching our eBPF own functions to the host nodes of the cluster, we can achieve in-depth observability for any workload running on it. This approach has the potential to greatly simplify and improve observability for microservices deployed on the cloud. We developed our proposed solution using open-sourced technologies and tested their performance and functionalities using chaos experiments. The entire Kubernetes deployment manifest for our proposed solution can be found on GitHub ¹.

1.7 Assumptions

To enable the repeatability of our work, we have made assumptions regarding the environment we will be using for our proposed solution. We assume that all our microservices will be running inside Containerd v1.6 containers. Containerd is one of the most popular container run-time engines, but other options exist, such as LXD, Docker, and Podman, which will not be considered. We also assume that the Kubernetes v1.26.0 orchestrator will be used to provision and manage our microservice containers in a distributed manner. These Kubernetes and Containerd versions work seamlessly with one another and reduce the time spent debugging errors or compatibility issues. The final assumption in our work is that all the host machines will be running Linux version 4.11 or greater. This version of Linux is required to leverage the eBPF CO:RE (Compile-Once Run-Anywhere) capability used in our provided eBPF functions.

¹<https://github.com/Howlla/cloud-observability>

1.8 Limitations of the Work

We have limited the scope of our research to distributed service mesh environments running web-based workloads. Web-based workloads include databases, web servers using Hypertext Transfer Protocol (HTTP), application interactions using Remote Procedure Calls (RPC) and load-balancing nodes. This research does not consider all the high-performance computing workloads like data mining, machine learning, artificial intelligence, and computer vision. We have also not considered monolithic architecture applications since they are becoming irrelevant in modern applications and do not benefit much from observability. Since we will use the open-source tool Grafana to analyze our observability data, we are limited to using Grafana-compatible data formats for transferring data. Our format is Open Telemetry, which aims to be the universal standard for observability data transmission using its OpenTelemetry Protocol (OTLP).

1.9 Delimitations

We can choose from various metrics when considering what to observe, but in this thesis, we will only consider performance metrics relevant to web-based workloads. The metrics we have identified to observe are:

1. **CPU Metrics** - This metric measures the percentage of time our CPU was running instructions from our workloads and observability processes.
2. **RAM Metrics** - This metric measures the percentage of RAM occupied by workload and observability processes.
3. **Network Metrics** - This metric is measured in the number of transmission packets per second that are flowing through our networking card.
4. **Requests Per Second** - This metric measures the number of successful responses our application can complete per second.
5. **I/O Latency** - This metric measures the time taken by our read and write requests to access the block storage device.

1.10 Organization of this Thesis

The rest of this thesis is organized as follows: Chapter 2 describes the background and related works. Chapter 3 is the design section, where the multi-layered design for observability and tracing framework is explained, data model and network topology is shown. Chapter 4 explains the experimental setup and results of our experimentation. In Chapter 5 we conclude the thesis and provide future directions for this work.

2. BACKGROUND AND RELATED WORK

This chapter defines the relevant background information for microservices, service mesh, and observability. Two major implementations discussed here are eBPF [14] and OpenTelemetry [15]. We also discuss related works and outline various challenges with current solutions.

2.1 Virtualization

Virtualization is a technology that enables the operation of multiple virtual instances of an operating system on a single physical machine. It provides the ability to create isolated virtual environments where various applications and services can operate independently from each other without any interference on each other’s resources. The importance of virtualization has risen significantly in the current era of cloud computing where it is essential to maximize the utilization of hardware resources to achieve cost-effective scalability and agility. Virtualization in cloud has led to major optimizations in networking and computing in the form of network function virtualization (NFV) and software defined networking (SDN) [16]. With more recent advancements in containerization and coordination technologies, many companies such as Netflix, Twitter, and Uber [17] are shifting away from monolithic architectures. The shift to microservices was largely due to their ease of management and development for large, highly-distributed, replicated applications. Monolithic architectures have several issues since their larger codebase is more difficult to debug, and we cannot independently change components without a cascading effect. Other issues include slower deployment times, difficulty scaling, dependency management, and tight coupling of components.

2.1.1 Containers

Containers have emerged as a more lightweight option compared to virtual machines (VM) due to their improved support for microservice architecture. As a result, the container market has grown significantly over the years, with its value projected to reach \$4.6 billion

by 2022 from \$762 million in 2016. [18]. They are the standardized method for microservices deployment, playing an vital role in fields such as service meshes, high-performance computing (HPC) and software defined networking (SDN) [19]. The main advantages of containers over virtual machines (VMs) lie in two areas. Firstly, containers can share resources and utilize the same operating system (OS) kernel, whereas VMs require their own individual copies [20]. Secondly, containers can be initiated and terminated almost instantaneously, whereas VMs require a significant amount of time to start [21]. In addition, containers have proven to be more effective than VMs in certain scenarios, such as microservices, because they are lightweight and do not necessitate a complete OS copy for each image. These benefits come at the cost of abstraction since almost all container components are virtualized. This results in monitoring and securing containerized applications that are harder than monolithic applications because containers are short-lived processes making it difficult to associate causes of errors with the container causing them. Also, by default, all containers in Docker run as root user. If the container process gets compromised, it can be used to gain lateral access and cause severe damage. Docker [22] is the most commonly used containerization framework to run containers. It combines Linux cgroups, namespaces, volumes, and `systemd` slices to isolate and concurrently run containers on a given host. Kubernetes uses it to run containers on nodes in an isolated manner so they cannot interfere with one another.

2.1.2 Container Orchestration Systems

A key technology used for managing containers is Kubernetes (K8s) [23]. Kubernetes divides our distributed cluster into 2 types of nodes, namely master nodes and worker nodes. The worker nodes are responsible for executing pods, while the master node orchestrates and schedules pods. Pods consist of a collection of co-located Docker containers tasked with performing a singular functionality. The master node runs a REST API that can dynamically adjust to manage and organize worker nodes as deployments and services. The `kubectl` API is a powerful tool that provides us with an interface to our cluster and configures it to provide optimal security. We can use `kubectl` to describe our cluster components and view debugging information, but this task is manual-intensive. Also, Kubernetes suffers from the

phenomena of hysteresis, i.e., the configuration we want our cluster to be in may not exactly be the current state, and there is the possibility of configuration drift. The full capabilities and features of `kubect1` are beyond the scope of this thesis, but more information can be found in the documentation [12].

Kubernetes [12] is a highly modular and extensible open-source container orchestration platform that automates containerized applications' deployment, scaling, and management. Initially developed by Google and now maintained by the Cloud Native Computing Foundation (CNCF), its architecture consists of interconnected components that work together to manage the lifecycle of containers and the resources they consume. Kubernetes has been architected to handle the complexity of distributed systems and the challenges of scaling and managing large containers.

Kubernetes architecture is divided into two components: the control plane and the worker plane. Control plane nodes act as the orchestrator for our worker nodes by scheduling, providing configuration, and monitoring the system's status. On the other hand, the worker nodes provide the container runtime responsible for running and managing the container's lifecycle. Unless specified otherwise, all application pods run inside our worker nodes as containers. The control plane node monitors the resource availability in our worker nodes and schedules pods for them. Kubernetes also provides Kubernetes Metrics Server, a metrics collection pipeline that can analyze metrics for autoscaling. Using Kubernetes Metrics Server for observability or any use case other than autoscaling is not advised.

2.1.3 Microservices

The microservices approach to software development, sometimes called a microservices architecture, divides functionality into multiple independent applications. Each independent application is much smaller than a monolithic program and only handles one function. To perform a task, the independent applications communicate over a network [24]. Microservices have emerged as a popular pattern for developing large-scale applications in cloud environments for their flexibility, scalability, and agility benefits. The cloud-native landscape comprises thousands of single-concern, loosely-coupled microservices running on container-

ized platforms like Docker. This approach can offer several benefits, including improved scalability, flexibility, and resilience. However, it also introduces new challenges, such as the need for effective service discovery and monitoring and the potential for increased complexity in testing and deployment [25].

One of the key challenges of microservices is ensuring observability, or the ability to monitor, debug, and diagnose issues within the system [26]. This is especially important in distributed systems, where failures or issues can occur in various parts of the system, and it can be difficult to trace their root cause. Because microservices are typically distributed and highly decoupled, traditional monitoring approaches may not provide full visibility into the system’s behavior [27]. Several approaches and tools have been developed for observability in microservices to address this challenge.

One approach is to use distributed tracing, which involves capturing and correlating trace data from individual service instances as they handle requests [28]. Distributed tracing can provide end-to-end visibility into requests across multiple services, allowing developers to quickly identify bottlenecks and diagnose issues. Tools like Zipkin [29] and Jaeger [30] have been developed to support distributed tracing in microservices.

Another approach is to use log aggregation and analysis, which involves collecting log data from multiple services and correlating it to gain insights into system behavior. Log aggregation can help identify patterns and anomalies in system behavior and provide a centralized source of truth for debugging and troubleshooting [31].

Finally, some approaches combine tracing and logging with metrics and other telemetry data to provide a more comprehensive view of system behavior. Metrics can provide insights into resource utilization, while telemetry data can help identify system-level issues like network latency and service availability [27]. Tools like Prometheus [32], and Grafana [33] have been developed to support this approach to observability in microservices.

Observability is a key challenge for microservices, but several approaches and tools have been developed to address it. Distributed tracing, log aggregation and analysis, and telemetry data are essential to providing insights into system behavior and help developers quickly identify and diagnose issues.

2.2 Network Service Mesh

Service mesh is an implementation improvement over microservices that helps to simplify the management of networking between microservices by providing a standardized approach that can be applied across multiple applications and environments. Additionally, a service mesh can help to address some of the challenges of microservices, such as observability and security [34]. A service mesh is an infrastructure layer designed specifically to manage communication between services in a system. Its primary responsibility is to ensure that requests are delivered reliably through the intricate network of services that make up a modern cloud-native application. The service mesh is typically implemented using a set of lightweight network proxies that are deployed alongside the application code, without requiring the application to have any knowledge of them [35]. As shown in Figure 2.1, a service mesh infrastructure logically consists of a data plane and a control plane. The data plane consists of a collection of intelligent proxies that are commonly deployed as sidecars [36]. These proxies regulate and oversee all network communication between microservices. With full visibility into each network packet, the data plane is responsible for service discovery, health checking, routing, load balancing, authentication/authorization, and observability. On the other hand, the control plane oversees and configures the proxies to manage traffic routing. Moreover, the control plane configures corresponding components to enforce policies and collect telemetry. In essence, the control plane serves as the central control hub or brain of a service mesh.

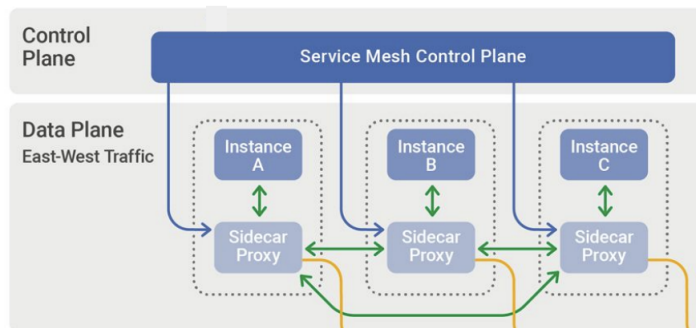


Figure 2.1. Sidecar proxy communication in service mesh networks [37]

2.2.1 Data Plane

In a microservice architecture, the data plane typically involves deploying sidecars alongside each service instance that requires communication with other services in the system. These proxies act as intermediaries for all requests and responses related to the service, and can handle a variety of tasks such as authentication, authorization, encryption, rate limiting, service discovery, logging, tracing, and load balancing [38], [39]. By implementing a service mesh, the network communication logic is separated from the application logic, effectively eliminating the need for developers to concern themselves with the intricacies of network communication. Instead, the service can interact solely with its local proxy. Additionally, a service mesh enables consistent implementation of network communication and related features across services, even when they are developed in different programming languages by different teams, without requiring duplication of effort. Sidecars are ideal standardized locations to collect observability data about the networking in our applications. It is important to note that the sidecars are present on the same machine as the service it is proxying, and they communicate over the loopback interface “localhost”. This does add some minor latency to our application, and we wish to avoid this if possible.

2.2.2 Control Plane

In microservice-oriented systems, which may consist of numerous services that are replicated multiple times for the sake of performance and reliability, a corresponding number of proxies must be managed. This number can easily reach dozens or even hundreds in larger systems. The control plane helps an administrator configure the behavior of all our proxies from a central API. The administrator can update configurations using YAML or REST calls to specify the routing, service discovery, health checking, load balancing, and observability needed in our service mesh architecture. Each data plane proxy must connect to the control plane to register itself and receive configuration details. The control plane is usually an independent node in our microservice mesh cluster with its own dedicated hardware. All the proxies pull the latest configurations from the control plane server and push the latest

observability data to the control plane. The control plane knows all the performance errors or latency of the distributed proxies report.

2.2.3 Ingress

The final component in a service mesh is the ingress gateway, which acts as a public entry point to the services in our mesh. The ingress gateway is an independent entity that implements Layer 7 and Layer 4 routing, sometimes called HTTP or path-based routing. This gateway determines which traffic enters the service mesh and what packets leave the service mesh, also known as North-South traffic. The ingress gateway plays a crucial role in distributed tracing in OpenTelemetry. We can associate unique trace IDs whenever a user tries to establish a new connection with the ingress gateway. This trace ID can then be propagated and recorded throughout the data plane to achieve complete observability in a service mesh. We leverage the Envoy Proxy [40] as the ingress load balancer for our solution, but other alternatives like Istio and Linkerd are also possible solutions.

2.3 Observability

Distributed architectures, by their very nature, are considerably more intricate, and present a multitude of operational difficulties concerning performance and availability. To address these challenges, we require observability data that can assist us in gaining a better understanding of these environments. Observability comes from control theory and measures how well a system’s internal states can be inferred based on external outputs. The more questions we can ask and answer about the system, the more observable it is. In the context of microservice, observability comprises three main data types: metrics, logs, and traces. Metrics are the variables that can be measured by looking at the system from the outside. Metrics are a numeric representation of data measured over intervals of time. Metrics can utilize mathematical modeling and prediction to derive knowledge of a system’s behavior over time intervals [26]. Since numbers are optimized for storage, processing, compression, and retrieval, metrics enable longer retention of data as well as easier querying. Logs are an immutable, timestamped record of discrete events that happened over time. Finally,

traces represent the interaction amongst multiple distributed applications and the ability to obtain a macro view of the request-response lifecycle. A single request may involve multiple services or components spread across different servers or geographical locations in a distributed system. Each of these services may generate its metrics, logs, and traces, but with a way to link them together, it can be easier to understand the end-to-end behavior of the system. A trace solves this problem by injecting a unique identifier (called a trace ID) for each request and propagating it across all the components involved in the request. As the request flows through the system, each service adds timing and other metadata to the trace, creating a detailed view of the entire transaction. This section defines the observability solutions in Kubernetes and their technical details.

2.3.1 Instrumentation for Observability

Traditional observability for monolithic applications depends solely on the programmer's capabilities to add additional code and import libraries that export observability data to be consumed. This approach requires the expensive operational overhead of programming and redeploying any additional observability functionality we wish to add. Another drawback of instrumenting our applications with observability code is the vendor lock-in and lack of interoperability between various libraries. Grafana's 2023 observability survey [41] highlighted that most organizations utilize 10-70 different libraries in their technology stack to monitor the various microservices in their organization. There is a growing need for a standardized data model that is vendor-agnostic and compatible with any technology stack.

2.3.2 Opentelemetry

OpenTelemetry (OTel) is a set of APIs, SDKs, tooling, and integrations designed to create and manage telemetry data such as traces, metrics, and logs. OTel is an open-source vendor-neutral standard incubated in the Cloud Native Computing Foundation (CNCF) formed as a result of the merger of OpenTracing and OpenCensus projects. It aims to define and standardize how telemetry data is received, processed, and stored from different sources. OTel provides libraries and SDKs which can be imported into your application; it

currently supports eleven (11) programming languages for observability. Some frameworks like Java Spring and Node Express require importing libraries to get basic observability in your applications. OTEL follows an agent-collector model where an OTEL agent is present inside the pod with our application. It communicates with a dedicated collector server using the Open Telemetry Protocol (OTLP) to share telemetry data. Some popular options for OTEL compatible collectors are Zipkin [29], and Jaeger [30].

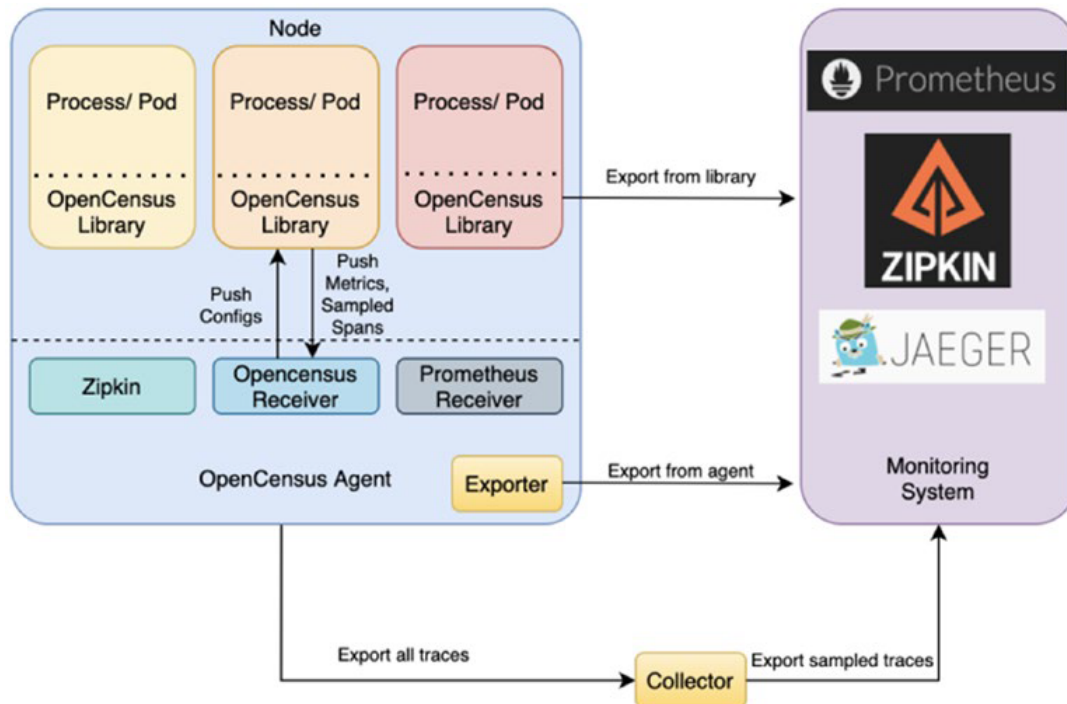


Figure 2.2. OpenTelemetry architecture

2.3.2.1 Collector

OpenTelemetry collector allows you to receive telemetry data from various sources and store it in a centralized location. It exists independently on a separate host inside our network and receives OTLP packets from agents. It just doesn't receive but also optionally transforms and sends the data further in the default configuration without making any specific changes to the core code. The collector features a unified code base accommodating metrics, logs,

and traces. Moreover, it assumes responsibility for retaining the gathered data in a time series database, facilitating later processing. Most collectors also provide a visualization dashboard, i.e., Jaeger [30] or Zipkin [29] as shown in Figure 2.2 to analyze the status of our application at a glance. For example, the OpenTelemetry [15] community defines the collector as:

“The OpenTelemetry Collector offers a vendor-agnostic implementation for receiving, processing and exporting telemetry data. In addition, it removes the need to run, operate, and maintain multiple agents/collectors to support open-source telemetry data formats (e.g., Jaeger, Prometheus, etc.) sending to numerous open-source or commercial back ends.”

Overall, the OpenTelemetry Collector is a vital component of the OpenTelemetry observability framework, providing a powerful and flexible way to collect, process, and export telemetry data from various sources to backends, such as tracing and metric systems, while adhering to established standards and conventions.

2.3.2.2 Agent

Opentelemetry agent standard defines the data model, protocol (OLTP), and payload adopted by many open-source solutions like Prometheus’ Node Exporter, Datadog agent, and New Relic Infrastructure agent are used for collecting and forwarding telemetry data from various sources to their respective monitoring and analytics platforms. To enable its operation alongside an application, an agent can be deployed as a binary, daemonset, or sidecar. It is typically installed on all hosts within the environment, and can enrich the telemetry data with metadata to facilitate customized queries. Additionally, the agent can handle activities such as batching, encryption, and compression, thus eliminating the need for individual client instrumentation and simplifying the process.

2.3.3 Prometheus

Prometheus [32] is a time series database created by SoundCloud in 2012 and was the second project to be incubated by CNCF after Kubernetes. Time series means that changes are recorded over time. Prometheus collects telemetry data from configured targets at given

intervals, evaluates rule expressions, displays the results, and can trigger alerts when specified conditions are observed.

One of the key features of Prometheus is its query language, PromQL, which allows users to retrieve and manipulate time series data collected by the Prometheus server. PromQL supports a wide range of operations, such as filtering, aggregation, and arithmetic transformations, and can be used to generate complex queries that provide insights into system performance and behavior.

Its data model is at the core of Prometheus, which defines how metrics are collected, stored, and queried. Prometheus uses a time-series data model, where each metric is represented as a series of timestamp-value pairs that can be queried and visualized over time. The data model also includes concepts such as labels and metrics types, which help to organize and categorize metrics.

Prometheus collector gets information by using per-node exporters deployed as Kubernetes DaemonSet. The exporters are HTTP servers that interpret the metrics of the target application or kernel and serve them as Prometheus-compatible “open metrics” on the `/metrics` endpoint. Several exporters help export existing telemetry data from third-party systems. In addition, exporters are helpful for cases where it is not feasible to instrument a given system with Prometheus metrics. The exporters that exist for Kubernetes and container metrics observability are described below.

2.3.3.1 Node Exporter Agent

The Prometheus Node Exporter is a software component that collects various system-level metrics, such as CPU usage, memory usage, disk I/O, and network statistics, from the operating system and makes them available for monitoring and alerting using the Prometheus monitoring system. The Node Exporter runs as a daemon on the target system, periodically collecting metrics and exposing them on an HTTP endpoint that a Prometheus server can scrape. The Prometheus can be configured to fetch telemetry data at a frequency of one second or greater.

Researchers have previously used the Node Exporter to monitor the performance of a containerized web application and identify performance bottlenecks [42]. The study found that the Node Exporter provided detailed insights into the application's resource usage and helped identify the root cause of performance issues. Another study used the Node Exporter to monitor the performance of a Hadoop cluster and optimize its configuration for better performance [43]. The study found that the Node Exporter provided valuable insights into the resource usage and bottlenecks of the cluster and helped identify configuration changes that improved its performance. Another study [44] used Node Exporter to monitor the performance of a Kubernetes cluster and identify performance issues in real-time. The study found that the Node Exporter provided a wealth of metrics data that allowed for fine-grained analysis of the cluster's resource usage. It helped identify and troubleshoot performance issues quickly. This data was then used to forecast the resource consumption of Kubernetes clusters based on historical metric data.



Figure 2.3. Node exporter dashboard [45]

2.3.3.2 cAdvisor Agent

cAdvisor (i.e., container Advisor) is an open-source tool developed by Google for monitoring containers' performance and resource usage. It collects and aggregates metrics such as CPU usage, memory usage, network usage, and disk I/O usage from containers and their associated host systems, providing a unified view of the performance of the entire container ecosystem. cAdvisor is deployed as a per-node daemonset and extracts telemetry data from the deployed microservices on the node using the Prometheus open-metrics format. The authors in [46] provide an overview of cAdvisor and its architecture and a detailed description of its capabilities and overhead. They compare Docker with other solutions in the market, like mpstats and docker stats, and conclude that cAdvisor takes more overhead but provides more granular and detailed information. The authors describe a comprehensive observability stack for containerized environments, which includes cAdvisor as a critical component for collecting performance metrics from containers. The authors argue that cAdvisor is essential for providing a holistic view of container performance and that it should be integrated with other observability tools, such as Prometheus and Grafana, for enhanced analysis and visualization.

Although agents like NodeExporter and cAdvisor help establish a baseline of observability for cloud applications, they are binaries that do not support customization and modifications easily. However, as all businesses are different, they all have different needs of observability, and eBPF offers us the programmability to modify our kernel easily and get more observability data out of it.

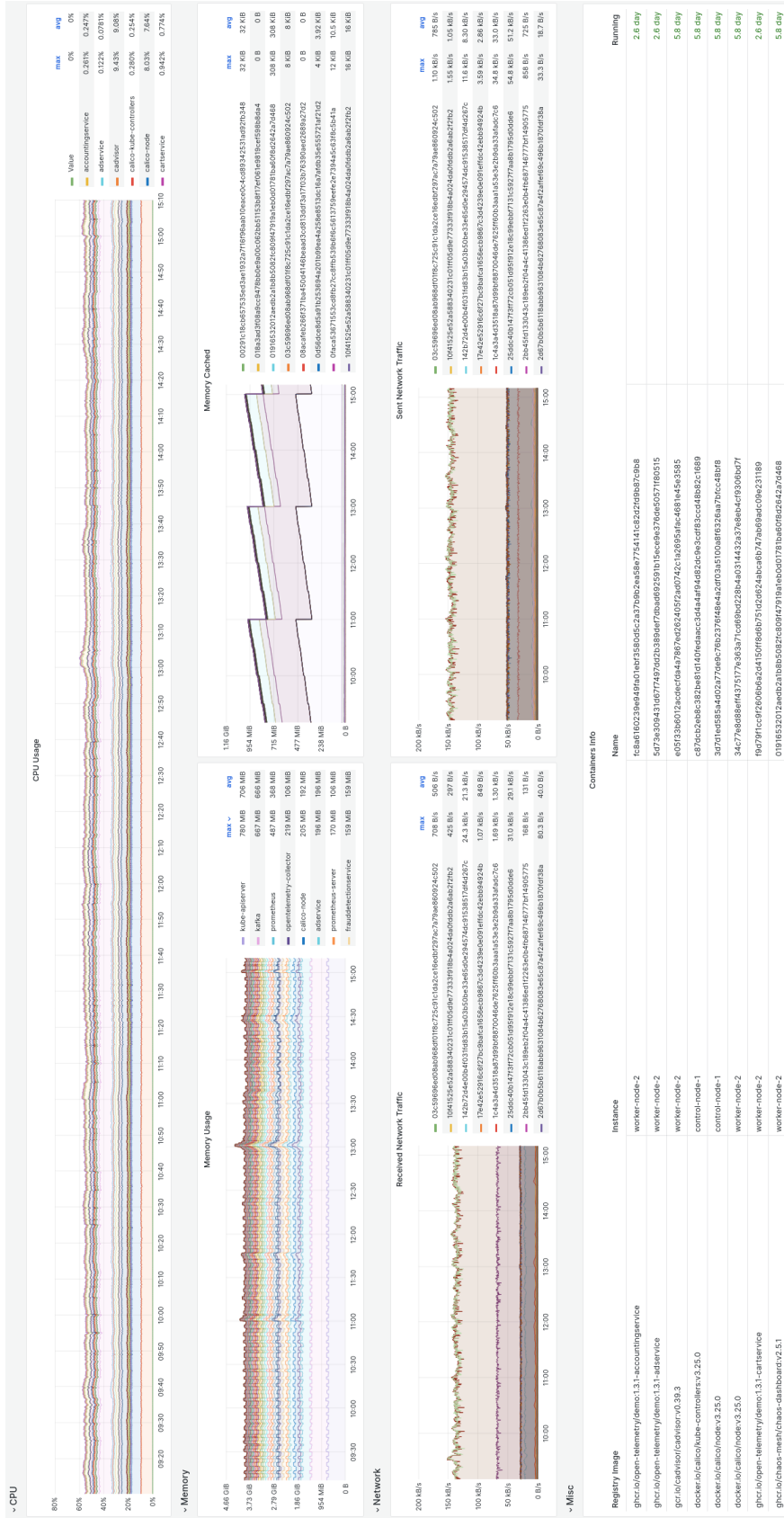


Figure 2.4. cAdvisor dashboard [47]

2.3.4 Extended Berkeley Packet Filter (eBPF)

eBPF has gained attention from many tech giants to improve their system performance and observability. Facebook’s Katran [48] uses eBPF for Layer-4 load balancing, TCP tuning, and DDoS protection. Google’s Adeb [49] provides an Android operating system with a Linux shell environment with tracing and high-security features. Netflix uses eBPF heavily in its networking stack for adding observability to the packets coming into its services. Cloud providers like AWS, Microsoft, and Google use eBPF for security [50], [51], network optimization [52], virtualization [53], and monitoring [54].

Linux Berkeley Packet Filters (BPF) have evolved over the past few years, making them the ideal tool for kernel tracing and metric collection. The original BPF patch [55] was added in 1992 for a specific purpose: to provide applications that monitor the systems network with direct information from the kernel. The `tcpdump` command in Linux was the first to use the packet filter. BPF introduced two significant innovations; first, a new VM was created inside Linux designed to work efficiently with the CPU register. Second was the introduction of a per-process buffer, which removed the need to copy data into the kernel before processing. The Extended Berkeley Packet Filter (eBPF) was added to Linux version 3.14 by Alex Starovoitov, improving the instruction set for the VM and increasing size to 64 bits and the number of registers to 10. The increase of the number and size of registers created new opportunities for programmers to develop more intricate software, as they could now transfer a greater amount of information through function parameters. One of the most popular service mesh projects to rise from eBPF is Cilium [56]. Cilium is an open-source project designed on top of eBPF to address the networking, security, and observability requirements of microservices workloads. By leveraging the programmability of eBPF, Cilium achieves high-performance observable networking in service meshes.

2.3.4.1 Architecture

BPF is a sophisticated virtual machine that executes code instructions in an isolated environment. BPF is compatible with compilers such as LLVM and GNU Compiler Collection (GCC), which enable the translation of C code into BPF instructions. Although these

instructions are similar to JVM bytecode in the Java programming language, they are Just-In-Time (JIT) compiled by the Linux kernel when called. The JIT will transform the BPF bytecode into machine code directly after verifying the program, avoiding this overhead on execution time. One powerful benefit of eBPF is that we do not need to restart our system to load BPF programs, then can be loaded on-demand dynamically.

Executing arbitrary code in the kernel can result in difficult bugs which cause the system to crash. To avoid this, the BPF verifier must verify the BPF bytecode. The conditions it checks for are:

- No infinite for loops should be present in the program.
- The size of the program should not exceed 4096 bytes.
- Program doesn't attempt unauthorized memory access outside the buffer provided.

Finally, the kernel must know when and where to run any BPF program, known as the probe or execution point where the program is attached. There are over 2000 probe locations in the kernel, and the list is growing [57]. The execution point can be a kernel function (*kprobe*), a function in your application (*uprobe*) or any arbitrary tracepoint in the Linux codebase.

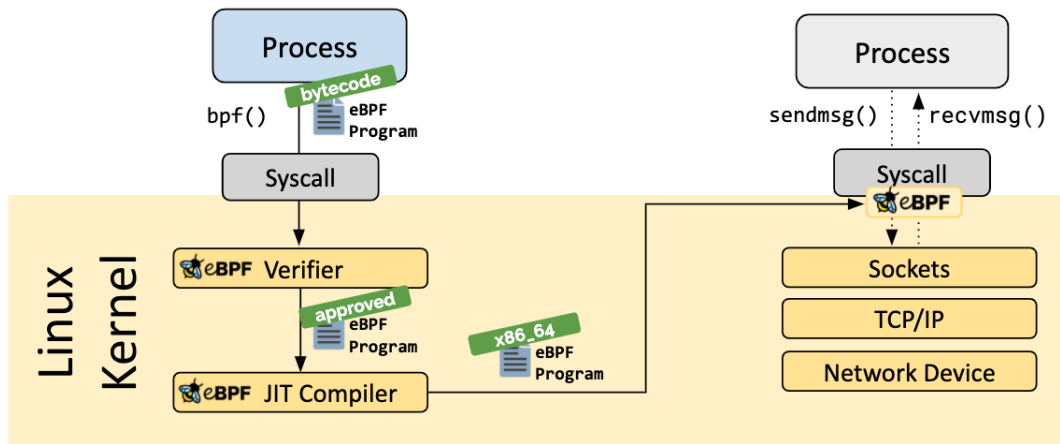


Figure 2.5. eBPF architecture

2.3.4.2 BPF Programming

Berkeley Packet Filter (BPF) programs are tiny programs that can be executed within the kernel to perform various network-related tasks; BPF programs can be written in C or assembly language and compiled into a bytecode format that the kernel can execute. BPF programs take the binary of the OS they are instrumenting as a dependency input to their program; this is required since the BPF compiler must understand how the memory layout is organized and what region of memory can be accessed. Figure 2.6 is an example of an eBPF program; the program filters network packets based on the source IP address and protocol type. The program takes a pointer to a `struct __sk_buff` structure as input, which contains information about the network packet being processed.

The program first extracts the Ethernet header and IP header from the network packet and checks to ensure they are within the bounds of the packet data. If either header is absent, the program returns a value of 0 to indicate that the packet should be dropped. Next, the program checks the source IP address of the packet to see if it matches a specific value (0x02020304 in this example). If the address does not match, the program returns a value of 0 to indicate that the packet should be dropped. Finally, the program checks the packet's protocol type to see if it is TCP. If the protocol is not TCP, the program returns a value of 0 to indicate that the packet should be dropped. If the packet passes all of these checks, the program can perform some action on the packet data (for example, it could update a packet counter or send the packet to another kernel module for further processing). In this example, the program returns a value of 1 to indicate that the packet should be accepted.

```

1  {
2  #include <linux/bpf.h>
3  #include <linux/if_ether.h>
4  #include <linux/ip.h>
5
6  int bpf_program(struct __sk_buff *skb) {
7      void *data = (void *) (long) skb->data;
8      void *data_end = (void *) (long) skb->data_end;
9      struct ethhdr *eth = data;
10     if (eth + 1 > (struct ethhdr *) data_end) {
11         return 0;
12     }
13     struct iphdr *ip = data + sizeof(*eth);
14     if (ip + 1 > (struct iphdr *) data_end) {
15         return 0;
16     }
17     if (ip->saddr != 0x02020304) {
18         return 0;
19     }
20     if (ip->protocol != IPPROTO_TCP) {
21         return 0;
22     }
23
24     return 1;
25 }
26

```

Figure 2.6. Example eBPF program

2.3.4.3 BPF Maps

The most useful data structure for exporting data from kernel space is the eBPF map. eBPF maps are a generic data structure for storing different data types. Data types are generally treated as binary blobs, so a user specifies the keys' and values' sizes at map-creation time. In other words, a key/value for a given map can have an arbitrary structure. Map types include hash tables or arrays, ring buffer, stack trace, least-recently-used, longest prefix match, and more. Figure 2.7 lists all the different types of maps supported by eBPF. Figure 2.8 shows the attributes needed when creating an eBPF map using the

BPF_MAP_CREATE system call. eBPF maps live in the kernel memory space but can be read and written to by both user and kernel space programs, making them very powerful programming tools. We will use eBPF maps to record time-series data from the host, and the collector will poll the maps to export data.

```
1  enum bpf_map_type {
2      BPF_MAP_TYPE_HASH,
3      BPF_MAP_TYPE_ARRAY,
4      BPF_MAP_TYPE_PROG_ARRAY,
5      BPF_MAP_TYPE_PERF_EVENT_ARRAY,
6      BPF_MAP_TYPE_PERCPU_HASH,
7      BPF_MAP_TYPE_PERCPU_ARRAY,
8      BPF_MAP_TYPE_STACK_TRACE,
9      BPF_MAP_TYPE_CGROUP_ARRAY,
10     BPF_MAP_TYPE_LRU_HASH,
11     BPF_MAP_TYPE_LRU_PERCPU_HASH,
12 };
```

Figure 2.7. eBPF map types

```
1  union bpf_attr {
2      struct { /* anonymous struct used by BPF_MAP_CREATE command */
3          __u32  map_type;          /* one of enum bpf_map_type */
4          __u32  key_size;          /* size of key in bytes */
5          __u32  value_size;        /* size of value in bytes */
6          __u32  max_entries;        /* max number of entries in a map */
7          __u32  map_flags;         /* prealloc or not */
8      };
9  }
```

Figure 2.8. eBPF map definition

2.3.4.4 libbpf

`libbpf` is a low-level library for working with eBPF programs and maps, providing a C API for loading, verifying, and manipulating eBPF objects. It is designed to be lightweight, portable, and easy to use in user-space programs. `libbpf` provides functions for loading eBPF bytecode from a file, verifying it, and loading it into the kernel. It can also load and verify eBPF maps, and data structures allowing eBPF programs to store and share data with user space. One of the critical features of `libbpf` is that it enables eBPF programs and maps to be compiled once and run anywhere, as long as the target system supports eBPF. When we compile an eBPF program with `libbpf`, it generates a binary object file containing the eBPF bytecode and map data and metadata about the eBPF objects. This binary object file can be loaded into the kernel on any system that supports eBPF. The eBPF program or map will execute as expected, regardless of the kernel version or system configuration. This CO:RE “compile once, run anywhere” approach makes it easy to develop and deploy eBPF programs and maps across multiple systems without worrying about kernel or system compatibility issues. `libbpf` is often a building block for higher-level eBPF tools like BCC.

2.3.4.5 BCC

BCC is another high-level toolkit that provides out-of-box eBPF program management and deployment. BCC uses Python to load, verify and manipulate eBPF objects. BCC comes with the LLVM compiler to convert Python code to BPF programs; therefore, there is more convenience at the cost of additional overheads. BCC provides multiple command-line tools for tracing and profiling Linux systems, such as `trace`, `profile`, `offcputime`, and `opensnoop`. These tools use eBPF to trace system calls, kernel functions, and other events and can provide detailed information about system performance and resource usage. BCC also offers multiple helper functions to write your own BPF programs, such as the `bpf_text` utility for writing eBPF programs in a higher-level language and the `bcc_perf` library for working with eBPF maps. Due to the additional overhead of compiling Python code to the target Operating System, BCC is not recommended for heterogeneous multi-node cluster environments.

2.3.4.6 Bpfttrace

Bpfttrace [58] is a tracing language for Linux that operates at a high level and is designed for use with the enhanced Berkeley Packet Filter (eBPF), which is available in recent Linux kernels (4.x). **Bpfttrace** leverages LLVM to compile scripts into BPF-bytecode and interfaces with the Linux BPF system and various Linux tracing capabilities, such as kernel dynamic tracing (kprobes), user-level dynamic tracing (uprobes), and tracepoints, via BCC. **Bpfttrace** was created to quickly debug production systems by running simple one-liners; it is not intended for real-time observability and monitoring of clusters. Table 2.1 provides a list of all the one-liners that **Bpfttrace** provides for kernel layer monitoring.

Table 2.1. Bpfttrace tools for kernel monitoring [58]

Tool	Description
<code>bashreadline.bt</code>	Print entered bash commands system wide.
<code>biolatency.bt</code>	Block I/O latency as a histogram.
<code>biosnoop.bt</code>	Block I/O tracing tool, showing per I/O latency.
<code>biostacks.bt</code>	Show disk I/O latency with initialization stacks.
<code>bitesize.bt</code>	Show disk I/O size as a histogram.
<code>capable.bt</code>	Trace security capability checks.
<code>cpuwalk.bt</code>	Sample which CPUs are executing processes.
<code>dcsnoop.bt</code>	Trace directory entry cache (dcache) lookups.
<code>execsnoop.bt</code>	Trace new processes via <code>exec()</code> syscalls.
<code>gethostlatency.bt</code>	Show latency for <code>getaddrinfo</code> / <code>gethostbyname</code> calls.
<code>killsnoop.bt</code>	Trace signals issued by the <code>kill()</code> syscall.
<code>mdflush.bt</code>	Trace md flush events.
<code>napttime.bt</code>	Show voluntary sleep calls.
<code>opensnoop.bt</code>	Trace <code>open()</code> syscalls showing filenames.
<code>oomkill.bt</code>	Trace OOM killer.
<code>pidpersec.bt</code>	Count new processes (via <code>fork</code>).

Table 2.1. Continued from previous page

Tool	Description
<code>runqlat.bt</code>	CPU scheduler run queue latency as a histogram.
<code>runqlen.bt</code>	CPU scheduler run queue length as a histogram.
<code>setuids.bt</code>	Trace the setuid syscalls: privilege escalation.
<code>ssllatency.bt</code>	Summarize SSL/TLS handshake latency as a histogram.
<code>sslsnoop.bt</code>	Trace SSL/TLS handshake, showing latency and return value.
<code>statsnoop.bt</code>	Trace stat() syscalls for general debugging.
<code>swapin.bt</code>	Show swapins by process.
<code>syscount.bt</code>	Count system calls.
<code>tcpaccept.bt</code>	Trace TCP passive connections (accept()).
<code>tcpconnect.bt</code>	Trace TCP active connections (connect()).
<code>tcpdrop.bt</code>	Trace kernel-based TCP packet drops with details.
<code>tcplife.bt</code>	Trace TCP session lifespans with connection details.
<code>tcpretrans.bt</code>	Trace TCP retransmits.
<code>tcpsynbl.bt</code>	Show TCP SYN backlog as a histogram.
<code>threadsnoop.bt</code>	List new thread creation.
<code>undump.bt</code>	Capture UNIX domain socket packages.
<code>vfscount.bt</code>	Count VFS calls.
<code>writeback.bt</code>	Trace file system writeback events with details.

2.4 Chaos Engineering

Chaos engineering is the process of intentionally introducing faults into a system to identify weaknesses, test failure scenarios, and enhance the system’s overall resilience. The objective is to conduct controlled experiments in a production environment to determine how a system responds to various failure scenarios, such as network outages, service failures, high-stress load or hardware crashes [59].

The main aim of chaos engineering is to improve confidence in a system’s ability to handle unexpected events and reduce the probability of downtime or other critical incidents. By proactively testing a system’s ability to handle failures, businesses can recognize potential vulnerabilities and improve their systems before real-world failures occur.

To implement chaos engineering, we must first identify its systems’ critical components and failure modes. Then we can then design experiments that simulate various failure scenarios and evaluate the system’s response to those failures. This could involve deliberately disrupting service traffic, terminating processes, or introducing latency or errors into the system.

Effective chaos engineering aims to ensure that experiments are well-designed, well-controlled, and do not have unintended consequences that affect the system’s stability. It is also crucial to have a clear understanding of the system’s architecture and dependencies, as well as monitoring and alerting systems to quickly detect and resolve any issues that may occur during chaos experiments.

In summary, chaos engineering is a potent tool for enhancing the resilience of complex systems and minimizing the risk of downtime or other critical incidents. However, it requires careful planning and execution to ensure that experiments are safe and effective and that the system remains stable throughout the process.

2.4.1 Chaos Mesh

Chaos Mesh [60] is a cloud-native Chaos Engineering platform that is open-source. It provides a wide range of fault simulation options and boasts extensive capabilities for orchestrating fault scenarios. With Chaos Mesh, developers can simulate various abnormalities that may occur in the real world during development, testing, and production environments, thus identifying potential problems in the system. To simplify the process of initiating a Chaos Engineering project, Chaos Mesh offers a dashboard Web UI that allows users to design their experiments.. Chaos mesh experiments are designed to be contained to certain pods or nodes in the Kubernetes cluster, allowing us to run tests safely without impacting all the services. Researchers have utilized chaos mesh to design experiments for their eBPF-

based networking protocol; they could properly identify the failing pods in the cluster and mitigate any damages [61]. Researchers have also built an anomaly detection system for Kubernetes cluster and found that Chaos mesh could correctly simulate anomalies that were picked up by their system [62].

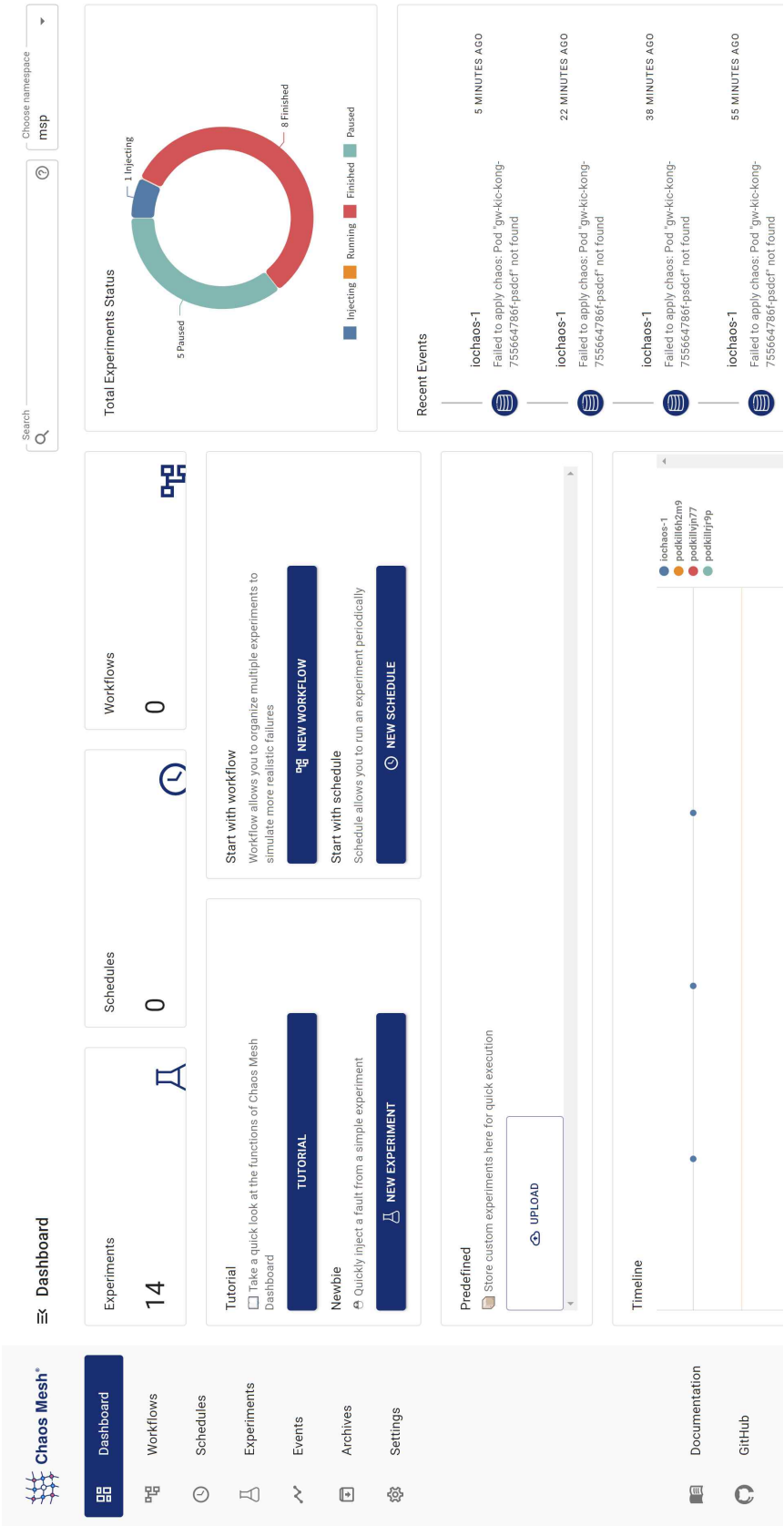


Figure 2.9. Chaos Mesh dashboard [60]

2.5 Related Works

In this section, some of the works in the area of eBPF and Kubernetes observability are discussed. ViperProbe [17], an eBPF-based deep metric collection tool. ViperProbe focused on improving metrics collection for service meshes by associating Process ID (PID) context with the metrics. This was an improvement of generic metrics monitoring which does not provide a breakdown of individual services running on a host. ViperProbe also includes an offline search paradigm for analyzing patterns to determine the minimal but effective set of metrics, i.e., CriticalMetrics, that enable run-time diagnosis of a service. Another research created a microservice tracing system based on Istio and Kubernetes [63], which required minimal code modification (auto instrumentation) as a result of this, and they even demonstrate that this type of tracing has little influence on the system performance. Their system monitors network packet headers and generates traces in a dashboard, but it does not consider other in-kernel metrics. Similarly, another group proposes MiSeRTrace [64], a kernel-level microservice visibility tool that utilizes eBPF to auto instrument tracing for individual pods in a service mesh at the kernel level. The authors in [54] use the extended Berkeley packet filter to monitor and profile performance in a distributed payments system. They conduct the profiling and tracing of several Interledger connectors using two full-fledged implementations of the Interledger protocol specifications [65]. To test the performance of their eBPF observability implementation [54] and [64] leverage DeathStarBench, a suite of ten sample microservices [66]. A recent study [61] introduced network monitoring at the kernel level for the Kubernetes cluster in Alibaba container service. Through the use of eBPF-based non-intrusive collection of user application Layer-7/Layer-4 layer network protocol interaction information, data collection of over 10 million throughputs per second can be achieved without any kernel or application code modification. Additionally, this approach has minimal impact on system application, less than 1 percent. The authors in [44] leverage Prometheus and NodeExporter for metric collection of Kubernetes workload; using the collected data, they trained a machine learning model to detect anomalies without any manual intervention automatically. A book [67] explains how eBPF can be leveraged for security of Kubernetes cluster by monitoring system calls and kernel traces to gain real-

time visibility into Kubernetes security. Finally, in [68] researchers focus on leveraging BPF programs for the observability of Linux systems; it provides details about all the kprobes and uprobes in the latest Linux kernel that can be leveraged for security and networking observability.

3. PROPOSED SOLUTION

This chapter describes the design strategies used for this work. The system design, data modeling, collector-agent design and data collection strategies using Prometheus timeseries database for our solution are explained in detail. The terms sidecar and agent are used interchangeably in this work.

3.1 System Design

This section discusses the system design chosen for multi-layer Kubernetes observability. For microservice management, Kubernetes orchestrator is used based on its advantages. In the cluster, all nodes will be connected to each other via a Kubernetes container network interface (CNI) which provides applications an abstracted view of the network infrastructure. One agent will be deployed per application pod and will export metrics and traces in the OpenTelemetry standard format to a Prometheus collector. Each node employs an eBPF exporter agent, cAdvisor agent and Node Exporter agent to export kernel metrics and traces to the Prometheus collector. The collector will merge the data received from the agents to build distributed traces and metrics and expose them to a Grafana dashboard. Figure 3.1 shows the system architecture to demonstrate the flow of data.

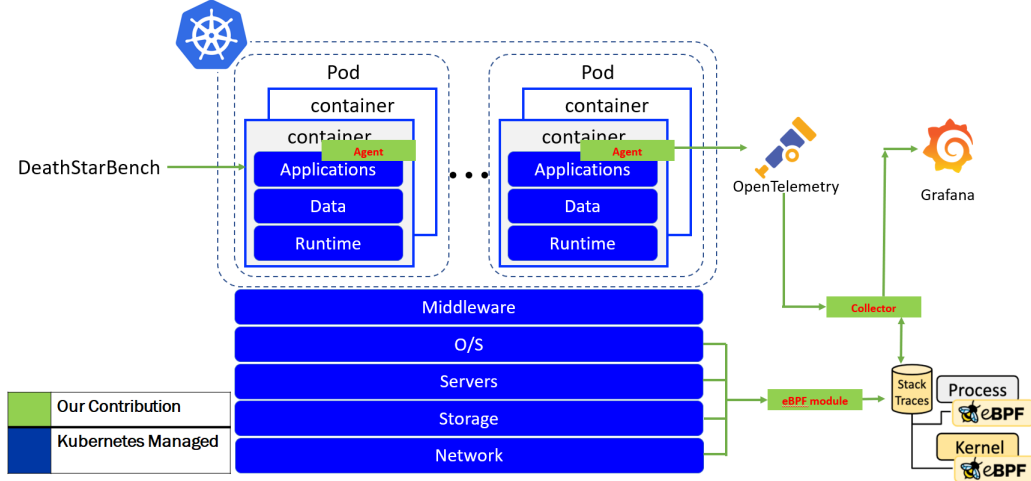


Figure 3.1. Proposed system design

3.2 Data Model

In an observability framework for service mesh, data plays a significant role in analyzing performance and identifying application causes of performance issues. The relationship between the observability data and the applications needs to be established. This section proposed a data model for Prometheus exporting agent data and eBPF-generated data to the collector.

The exporter agents use the OpenTelemetry Protocol (OLTP) to export L7, L4 and L3 application metrics and traces. OLTP uses the Span Model inspired by Dapper [28] to generate traces. The request is forwarded using a unique traceId injected at the ingress gateway to represent a single request-response cycle consisting of multiple individual spans for each microservice. Figure 3.2 is an example trace from the OpenTelemetry documentation.

```
1  {
2      "name": "Hello-World",
3      "context": {
4          "trace_id": "0x5b8aa5a2d2ax72e8321cf37308d69df2",
5          "span_id": "0x5fb397bevzd26b51",
6      },
7      "parent_id": "0x411581bf3cb55c13",
8      "start_time": "2022-05-29T18:52:58.114304Z",
9      "end_time": "2022-05-29T18:52:58.114435Z",
10     "attributes": {
11         "http.route": "/path1"
12     },
13     "events": [
14         {
15             "name": "hello world!",
16             "timestamp": "2022-05-29T18:52:58.114561Z",
17             "attributes": {
18                 "event_attributes": 1
19             }
20         }
21     ],
22 }
```

Figure 3.2. OTLP trace example

In Figure 3.2 note the context attribute that correlates spans across service and process boundaries. For instance, if a service call is made from Service A to Service B, and there is a need to monitor this transaction in a trace, OpenTelemetry utilizes trace context to capture the ID of the trace and current span from Service A. This allows spans that are created in Service B to connect and contribute to the trace. The process of capturing trace context and connecting spans to a trace is known as context propagation, which is the core concept that enables distributed tracing. Through context propagation, spans can be correlated and assembled into a trace, regardless of where they are generated. Spans represent units of work or operation and are the building blocks of traces. The span contains the start and end timestamps for a request, allowing us to calculate the total time taken for processing. Figure 3.3 demonstrates how spans are linked to complete a trace.

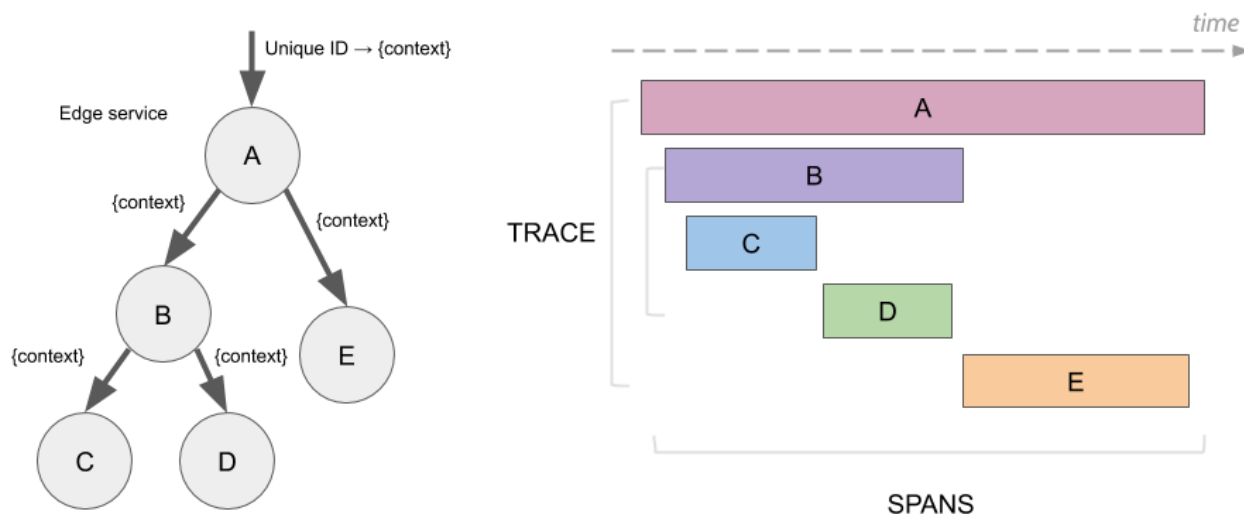


Figure 3.3. Span-trace data model [30]

3.3 Application Layer Observability

The application is the first level of observability and is achieved by leveraging the OpenTelemetry libraries to instrument our application. OpenTelemetry is a powerful open-source observability framework that allows you to collect telemetry data from various sources, including applications, infrastructure, and network components. With OpenTelemetry, you

can easily capture, trace, and analyze data across multiple distributed microservices by adding additional code or instrumenting the codebase.

We leveraged the OpenTelemetry library to instrument our demo application, Table 3.1 describes all the microservices deployed in our cluster and their respective programming language used. All these microservices had modifications to export OTel-compatible metrics and traces. Figure 4.9 provides a graph diagram of the networking between the microservices.

By importing the appropriate language SDK in our project, we can make our code generate metrics, logs, and traces. This requires tedious effort from the application developer but is the most powerful in developing custom business-specific observability data. One of the most potent features of OpenTelemetry is its ability to create application traces. In our solution, OpenTelemetry is used to export the metrics and traces generated by our applications. Our solution collects and orders all this data correctly using the Prometheus time-series database. Since each application generates observability data independently, we must have synced clocks in all our nodes; Kubernetes provides a managed network time protocol (NTP) server for nodes to communicate and sync with.

Table 3.1. Microservices in our test application

Microservice	Description
Accounting Service	Calculates the total of sold products (Go)
Ad Service	Serves Ads to users (Java)
Cart Service	Records shopping cart for users (C#)
Checkout Service	Process a checkout order from the user (Go)
Currency Service	Convert different currency amounts (C++)
Email Service	Sends order confirmation email (Ruby)
Frontend	Provides HTTP accessible UI (TypeScript)
Load Generator	Locust.io based load generator (Python)
Payment Service	Processes credit card information (JavaScript)
Product Catalog Service	Return information about products (Go)
Quote Service	Calculates shipping costs for orders (PHP)
Recommendation Service	Returns a list of recommended products (Python)
Shipping Service	Provides tracking information of order (Rust)

3.4 Network Observability Agents

The service mesh sidecars of modern microservices provide the perfect probe point to observe our network. There are two types of networking traffic in cloud-native microservices, i.e., North-South and East-West. North-South traffic refers to the communication packets to-and-from clients outside our Kubernetes clusters. East-West traffic, conversely, refers to packets that flow between microservices or nodes within a network.

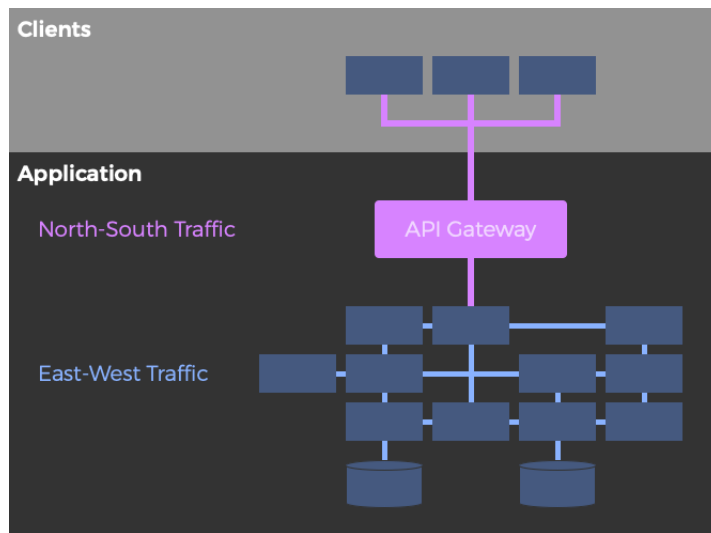


Figure 3.4. Types of network traffic in the cloud [69]

The Sidecar approach allows us to monitor North-South traffic by using an ingress API gateway for all incoming network traffic. Although North-South traffic is crucial in understanding the inflow of traffic, it does not give us any detailed contextual information.

To achieve contextual observability of network traffic [28] proposed distributed trace IDs. Whenever a packet comes to the API gateway, we can attach headers to the packets propagated throughout the network. The sidecar injects distributed trace IDs as headers which are extracted using OpenTelemetry and use to create spans for visualization, as shown in Figure 3.3. Using the distributed traces and timestamps, we can achieve East-West Observability with slight modification to the default ingress gateway.

For our solution, we leveraged [40] since it automatically injects distributed traces for East-West network packets and provides metrics for North-South network packets.

3.5 Node Observability Agents

Finally, we must deploy an agent for every node in our cluster to achieve host-level observability. AWS uses this approach in their CloudWatch agent and by GCP in their Stackdriver agent. Most host-level observability agents are root-level applications that can access the `/proc` virtual file system. The `/proc` folder doesn't contain real files but runtime system information [70] (e.g., system memory, devices mounted, hardware configuration, etc.). For this reason, it is regarded as a control and information center for the kernel. Prometheus's NodeExporter is a wrapper that reads data from the `proc` folder and serves it at an HTTP endpoint. cAdvisor is a tool from Google that exports Prometheus metrics like NodeExporter but provides the breakdown of resource utilization per container, i.e., it is a container-aware exporter. Exporters like NodeExporter and cAdvisor provide time series metrics by sampling the data in the `/proc` folder. Sample means that once every specified time interval (minimum 1 second), Sampling can cause us to lose information [71]. Still, it is necessary to reduce the amount of storage and network usage for monitoring.

Using cAdvisor and Node Exporter, we can obtain baseline telemetry information about CPU Utilization and RAM Utilization at the node level (Node Exporter) and the pod level(cAdvisor). This information is useful for getting up and running with metrics, but both of these tools do not allow custom metrics to be exported outside their scope. Also, since they are sampling the information, it is possible to lose the granularity of the data. A high resource consumption but short lifespan process might be running in our cluster, but these tools would not pick up such a process. Therefore, we need custom metric information from our nodes that can run without adding much overhead and not use sampling to get complete information.

To create custom metrics at the host level, we leveraged the eBPF exporter from Cloudflare. The eBPF exporter works by running eBPF programs within the kernel of a Linux system and collecting metrics directly from the kernel. These metrics may include system performance, resource utilization, network traffic, and more information. The metrics are written to an eBPF Map, and all the events are captured without dropping any information. eBPF exporter will then poll the eBPF maps to fetch kernel-level traces from the host. Us-

ing the tuple of process ID (PID) and traceID as unique identifiers, it will join the two data sources and save them in the Prometheus time-series database. The most recent data will also be immediately exported to a Grafana dashboard for faster debugging of performance issues. eBPF exporter created ten custom metrics by writing our own eBPF exporter scripts and was able to visualize their data on Grafana. The benefit of using eBPF exporter is the extremely low overhead, less than 1 percent of NodeExporter [72], and the fact that it does not perform sampling, providing complete detailed information. Using eBPF, we can access metrics at the Kernel level that are not accessible by any other tool. Table 3.2 lists the metrics we monitor using custom eBPF scripts deployed on all nodes. There are over 2000 possible metrics and tracepoints available for our kernel version, but these provide a good estimation of the capabilities of eBPF.

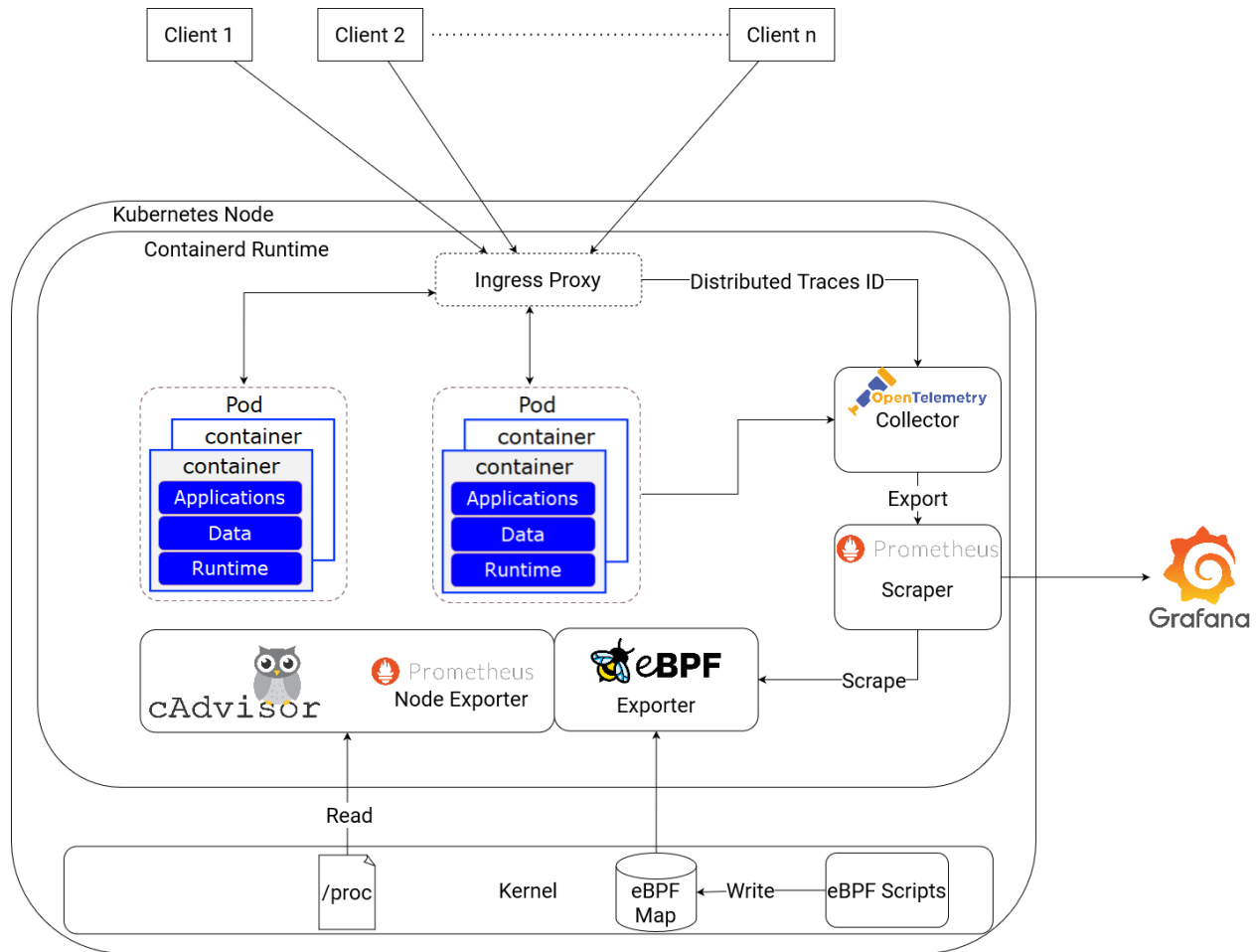


Figure 3.5. Proposed solution agent-collector architecture

Table 3.2. eBPF scripts implemented for real-time kernel observability

Script	Description
<code>accept-latency.bpf.c</code>	Latency for duration sockets sit in accept queue
<code>biolatency.bpf.c</code>	Block I/O latency
<code>bitesize.bpf.c</code>	Block I/O size
<code>cachestat.bpf.c</code>	Number of Page cache operation per cgroup
<code>llcstat.bpf.c</code>	Number of Last level cache operations per cgroup
<code>malloc.bpf.c</code>	Number of malloc calls per cgroup
<code>oomkill.bpf.c</code>	Number of oomkills per cgroup
<code>runqlat.bpf.c</code>	Latency while process waits in schedulable state
<code>tcpbacklog.bpf.c</code>	TCP SYN backlog size

3.6 Prometheus Collector

The collector in our architecture is the key system responsible for combining application, network, and host layer metrics and traces. The collector is deployed once per cluster, communicating with all the individual agents and polling eBPF maps. Prometheus is written in GoLang, a lightweight system programming language that can leverage multi-threading to achieve high throughput. The Prometheus collector will initially download a configuration from `kubectl` for service discovery of the pods in the node. We used three Host level exporters to get complete observability at the host level. `NodeExporter` provides us with an overview of resource consumption at a per-node level. `cAdvisor` helps us dive deeper and identify the containers that are not behaving as expected. Finally, the eBPF exporter allows organizations to customize and deploy thousands of metric collectors to help debug real-time issues. For network layer observability Prometheus communicates with the Envoy ingress API gateway to periodically get data about network performance in the cluster. Finally, for the application layer metrics, logs and traces, Prometheus polls the OpenTelemetry deployment in our cluster to get information.

3.7 Evaluation Framework

To successfully answer the research questions, we have to devise a method to assess the capabilities of our proposed multi-layered observability solution in a cloud environment rather than on our local machine. Testing and benchmarking require a computing platform that is not virtualized since virtualization causes inconsistent resource consumption [73]. We will leverage the bare-metal nodes provided by [74] to ensure repeatability and remove any noise added due to virtualization in a shared tenant environment. Bare-metal nodes are physical hardware servers that can be leased for a short time. They do not have any virtualization and allow us to create the most consistent and repeatable tests. We use the Chameleon TACC baremetal platform to simulate our Kubernetes benchmark. We reserved three Intel(R) Xeon(R) CPU E5-2670 baremetal instances with 128GB of RAM and InfiniBand networking. The Operating System used is 64-bit Ubuntu 22.04, but any OS with Linux kernel 4.15 or greater will work with the eBPF examples shared. Containerd version v1.6.12 is the container runtime, and Kubernetes version v1.25.6. A 100 GB NFS share stores the Prometheus metrics generated in our cluster.

The service mesh workloads to be run on our bare-metal testbed are borrowed from OpenTelemetry’s E-commerce Demo, a popular collection of different microservices created for benchmarking and testing. The demo comes with 10 microservices as described in Table 3.1 that already come with application layer observability instrumented into it using OpenTelemetry SDKs and libraries. We deploy the application on our Kubernetes cluster with our observability agents running to monitor the application. We created custom Grafana dashboards to analyze our proposed solution’s capabilities and answer **RQ1** and **RQ2**.

To answer **RQ3**, we need to inject anomalies into our Kubernetes cluster and monitor our Grafana dashboard to see whether our tools can correctly identify the anomaly. We leveraged two tools to inject these anomalies. Locust.io a synthetic load generation framework, was used to simulate real-world users using browsing our E-commerce platform. Using locust.io, we can simulate thousands of simultaneous users swarming our application by making HTTP calls to various services. Figure 3.6 shows the locust UI that can simulate users. In a

cloud environment, there can be other forms of anomalies other than just a high load on the application. Pods may fail, software like crypto miners can consume the resources for malicious purposes, our DNS could stop working etc. To simulate these kinds of anomalies for testing, we leveraged the chaos mesh open-source libraries to inject these anomalies.

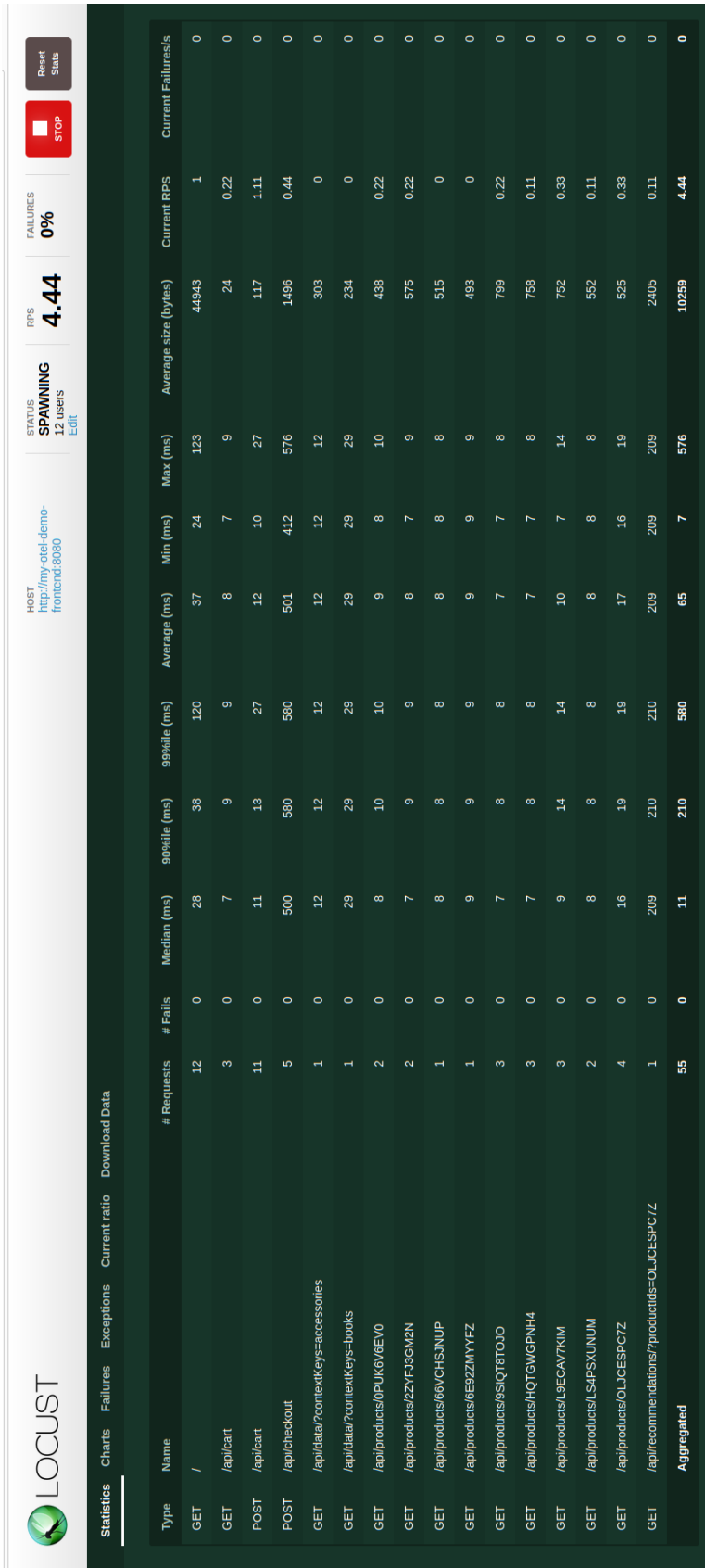


Figure 3.6. Locust load testing

4. EXPERIMENTAL RESULTS

This chapter provides a tutorial to deploy the proposed solution on the Chameleon cloud and explains the testing results. Testing uses chaos experiments designed to specifically target the microservices deployed in our cluster. Due to Chameleon’s advanced networking and computational capabilities, it was challenging to saturate the resources without synthetic load generated by chaos experiments. The impact of these experiments is observed on the Grafana dashboard visualized by us.

4.1 Experiment Testbed

This section details deploying our cloud-observability implementation on a baremetal cluster on the chameleon cloud platform [74]. The source code be found at www.github.com/howlla/cloud-observability.

4.1.1 Baremetal Resource Reservation

We can log in to Chameleon Cloud using a recognized university’s federated login (e.g., Boilerkey); next, we must request a lease for our required resources. Leases for baremetal instances are limited to a maximum duration of 7 days, and we must renew the lease before it expires to maintain our work. For the compute, we choose `haswell_ib` since they are abundantly available for cluster configurations due to being one generation older. We must reserve 3 hosts to deploy a highly-available Kubernetes cluster; this is important as our testing may crash some pods. Although this is sufficient for running our cloud-observability toolkit and visualizing observability data on a Grafana dashboard, we are still missing one thing. To successfully achieve complete observability, we must store historical observability data. Using the NFS storage service provided by Chameleon, we can share observability data from all sources into a single storage location. To gain access to NFS storage from our baremetal instance, we must deploy a network of `usage_type` of `storage`.

Create Lease

Create Lease

General * Hosts Networks

Lease Name *

MyLease

Start Date ?

Today

Start Time ?

Now

Lease Length (days) ?

7

Ends ?

2023-04-09

End Time ?

Same time as now

Your timezone is currently configured as UTC. If you need to update your timezone please go to your [User Settings](#).

Please be courteous to other users of the testbed and make sure your lease represents a responsible use of Chameleon resources and complies with our [best practices](#). Chameleon operators reserve the right to terminate leases judged to be abusive.

For leases shorter than 24 hours, use a lease length of zero days.

Reserve Hosts

Minimum Number of Hosts ?

3

Maximum Number of Hosts ?

3

Resource Properties ?

node_type

=

compute_haswell_ib

For specific node reservations, you can find the node UUID using Resource Discovery on the user portal.

Add Filter

Cancel

« Back

Next »

Cancel

« Back

Next »

Figure 4.1. Requesting lease on the chameleon cloud

Create Lease

General * Hosts Networks

Reserve Network

Network Name ?

StorageNetwork

Network Description ?

Resource Properties ?

physical_network

=

physnet1

usage_type

=

storage

Add Filter

Reserve Floating IPs

Number of Floating IP Addresses Needed ?

0

Network name is required when reserving a network.

Network reservations are required for some advanced networking features, like stitching and the filesystem. It is possible to create a private layer 2 VLAN without a network reservation. [Learn more](#).

Cancel

« Back

Create

Figure 4.2. Storage network for NFS on the Chameleon cloud

62

Launch Instance

Details

Source

Networks

Network Ports

Key Pair

Please provide the initial hostname for the instance, the availability zone where it will be deployed, and the instance count. Increase the Count to create multiple instances with the same settings.

Project Name

CHI-210916

Instance Name *

Control-Plane-1

Description

Reservation *

MyLease (95d45d60-0e99-4162-913c-59c531c9c47f)

Count *

1

Total Instances (No Limit)

8 Current Usage

1 Added

Cancel

Show Advanced

Back

Next >

Launch Instance

(a) Input instance name and specify lease to use

Launch Instance

Details

Source

Networks

Network Ports

Key Pair

Instance source is the template used to create an instance. You can use an image, a snapshot of an instance (image snapshot), a volume or a volume snapshot (if enabled). You can also choose to use persistent storage by creating a new volume.

Select Boot Source

Image

Allocated

Displaying 1 item

Name	Chameleon Supported	Updated	Size	Format
> CC-Ubuntu22.04	Yes	3/1/23 2:14 AM	1.73 GB	QCOW2

Displaying 1 item

(b) Select boot image for operating system

Launch Instance

Details

Source

Networks

Network Ports

Key Pair

Networks provide the communication channels for instances in the cloud.

Allocated 1

Select networks from those listed below.

Network	Subnets Associated	Shared	Admin State	Status
> 1 StorageNetwork	StorageNetwork-subnet	No	Up	Active

Available 1

Select at least one network

Click here for filters or full text search.

Network	Subnets Associated	Shared	Admin State	Status
> sharednet1	sharednet1-subnet	Yes	Up	Active

(c) Select storage network to connect to NFS

Figure 4.3. Launch a baremetal instance on Chameleon cloud

63

4.1.2 Launching Instances

After successfully creating our lease, we can now create our Kubernetes cluster. Using the Compute → Instances tab on the Chameleon dashboard, we can navigate to the dashboard to manage our instances. We must use the configurations described in Figure 4.3 to instantiate our one control node and two worker nodes.

4.1.3 Configuring Kubernetes with cloud-init

The cloud-init script for automated installation of Kubernetes in a Linux Ubuntu 22.04 cluster is provided in Figure 4.5. The Chameleon Cloud UI interface can be seen in Figure 4.4.

Launch Instance

You can customize your instance after it has launched using the options available here. "Customization Script" is analogous to "User Data" in other systems.

Load Customization Script from a file

No file selected.

Customization Script (Modified) Content size: 1.99 KB of 16.00 KB

```
#cloud-config
runcmd:
- apt-get update
- hostname control-node-1
- hostnamectl set-hostname control-node-1
- swapoff -a
- |
  cat <<EOF | tee /etc/modules-load.d/k8s.conf
```

Disk Partition

Automatic

☐ Configuration Drive

Figure 4.4. Using a cloud-init script to configure a baremetal Kubernetes cluster

While launching our instances, we can leverage cloud-init scripts to auto-set up our Kubernetes cluster during the bootup sequence of our nodes. Using cloud-init can help simplify deploying and configuring cloud instances since it automates many initial setup tasks that would otherwise need to be performed manually. We can navigate to Advanced options in

the chameleon dashboard and upload our cloud-init script for the control plane found at <https://github.com/Howlla/cloud-observability/blob/main/cloud-init-scripts/cp.txt>. Similarly, the wp.txt can be used when configuring the worker or data plane nodes.

We have described the crucial commands in our script that help set up our Kubernetes cluster below.

- Line 4: This disables swap memory on the system. This is required for Kubernetes to function.
- Lines 5-16: This requires kernel modifications to support overlay networking between Kubernetes pods.
- Lines 17-23: This installs `containerd`, the container runtime used by us. This also modifies the `/etc/containerd/config.toml` file to enable systemd cgroups.
- Lines 24-29: Installs version 1.25.6 of `kubeadm`, `kubect1` and `kubelet`. These are all components of Kubernetes management and runtime.
- Lines 31-33: These commands create a Kubernetes configuration file for the current user by copying the configuration file for the admin user to the current user's home directory. The `sudo chown` command sets the ownership of the configuration file to the current user.
- Lines 35-36: These commands download the Calico network policy controller YAML file and apply it to the Kubernetes cluster using `kubect1`. Calico is a popular networking solution for Kubernetes clusters, and it is used here to enable network communication between pods.

```

1  sudo apt-get update
2  sudo hostname control-node-1
3  sudo hostnamectl set-hostname control-node-1
4  swapoff -a
5  cat «EOF | sudo tee /etc/modules-load.d/k8s.conf
6  overlay
7  br_netfilter
8  EOF
9  sudo modprobe overlay
10 sudo modprobe br_netfilter
11 cat «EOF | sudo tee /etc/sysctl.d/k8s.conf
12 net.bridge.bridge-nf-call-iptables = 1
13 net.bridge.bridge-nf-call-ip6tables = 1
14 net.ipv4.ip_forward = 1
15 EOF
16 sudo sysctl -system
17 sudo apt-get install -y containerd
18 sudo systemctl start containerd
19 sudo mkdir -p /etc/containerd
20 sudo containerd config default | sudo tee /etc/containerd/config.toml
21 sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' \
22 /etc/containerd/config.toml
23 sudo systemctl restart containerd
24 VERSION=1.25.6-00
25 sudo apt-get install -y kubelet=$VERSION kubeadm=$VERSION \
26 kubectrl=$VERSION
27 sudo apt-mark hold kubelet kubeadm kubectrl containerd
28 sudo systemctl enable kubelet.service
29 sudo systemctl enable containerd.service
30 sudo kubeadm init -kubernetes-version v1.25.6
31 mkdir -p $HOME/.kube
32 sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
33 sudo chown $(id -u):$(id -g) $HOME/.kube/config
34 wget https://raw.githubusercontent.com/projectcalico/calico/v3.25.0/ \
35 manifests/calico.yaml
36 kubectrl apply -f calico.yaml

```

Figure 4.5. cloud-init script for the control plane

4.2 Cloud-observability Framework

Once our Kubernetes cluster has been set up, we can deploy the proposed multi-layered observability framework. We have created Kubernetes manifests and Dockerfiles for all our deployments to simplify recreating our experiments. We can SSH to our control plane node using an SSH client like PuTTY. Once we have connected our control plane node, we can clone the repository by running `git clone www.github.com/howlla/cloud-observabilty` in our terminal.

4.2.1 Prometheus Collector

The first component we must instantiate is the Prometheus time-series database. Since we are using an NFS share to store our observability data, we must mount the share to `/mnt` directory on the nodes by running the command “`sudo mount -t nfs -o nfsvers=4.2,proto=tcp <export location path> /mnt`”

First, we create a Kubernetes namespace named “monitoring” to deploy all our observability cluster components. This can be done by running “`kubect1 apply -f monitoring-namespace.yaml`”

```
1  apiVersion: v1
2  kind: Namespace
3  metadata:
4    name: monitoring
```

Figure 4.6. Kubernetes YAML manifest for monitoring namespace

To deploy the Prometheus server, we have to run “`kubect1 apply -f prometheus/`”. It should deploy the Prometheus server and use the mount location `/mnt` to store all observability data. Prometheus uses a configuration map to understand which exporters are to be scrapped. We have already set up the `prometheus/config.yaml` to scrape for Node Exporter, cAdvisor, eBPF exporter, Envoy sidecars, and OpenTelemetry collectors. Still, if you

wish to use other exporters or agents for observability, you can edit the `prometheus/config.yaml` file to add those exporters.

4.2.2 Grafana Dashboard

We will leverage the Grafana dashboard to visualize and analyze the observability data generated. To deploy Grafana, we can run “`kubectl apply -f grafana.`” Once deployed, we can access the Grafana dashboard by SSH tunneling to port 32000 on our cluster using PuTTY. Grafana should automatically detect the Prometheus data store in our cluster and start fetching real-time data from Prometheus. In case you wish to use a different collector than Prometheus to store observability time series data, you can follow the steps below.

1. Open the Grafana web interface and log in using the default admin credentials.
2. Click on the *Configuration* gear icon on the left-hand menu, and select ‘Data Sources’.
3. Click on the *Add data source* button in the top-right corner of the screen.
4. Select *Prometheus* or any other database from the list of data sources.
5. In the HTTP section, enter the URL for the Prometheus instance, such as `http://prometheus-service.monitoring.svc:8080`.
6. Enter a name for your data source, such as *Prometheus*.
7. Click the *Save & Test* button to verify the connection to the Prometheus instance.

4.2.3 Agents

Next, we will deploy Agents used to achieve host-level and kernel-level observability from our application. We identified three agents for our solution: Node Exporter, cAdvisor, and eBPF exporter. We have simplified deploying these exporters by utilizing Kubernetes manifest files. To deploy Node Exporter and cAdvisor, we have to run two commands:

```
kubectl apply -f node-exporter
kubectl apply -f cadvisor
```

```

1  cd cloud-observability/ebpf/examples
2  make build
3  #If youre having trouble building on the host, try building in Docker:
4  docker build -t ebpf_exporter .
5  docker cp $(docker create ebpf_exporter):/ebpf_exporter ./
6  #Run the subset of eBPF scripts you want
7  sudo ./ebpf_exporter -config.dir examples -config.names accept-latenc
8  sudo ./ebpf_exporter -config.dir examples -config.names biolatenecy
9  sudo ./ebpf_exporter -config.dir examples -config.names bitesize
10 sudo ./ebpf_exporter -config.dir examples -config.names cachestat
11 sudo ./ebpf_exporter -config.dir examples -config.names llcstat
12 sudo ./ebpf_exporter -config.dir examples -config.names malloc
13 sudo ./ebpf_exporter -config.dir examples -config.names oomkill
14 sudo ./ebpf_exporter -config.dir examples -config.names runqlat
15 sudo ./ebpf_exporter -config.dir examples -config.names tcpbacklog
16 sudo ./ebpf_exporter -config.dir examples -config.names pgflt

```

Figure 4.7. Deploying eBPF exporter

To deploy the eBPF exporter, we must complete a few additional steps first. Using a Linux distribution other than Ubuntu 22.04, first requires compiling all the BPF scripts for the correct OS. Figure 4.7 describes the commands you must run to successfully compile and deploy the eBPF exporter on our cluster. Note that eBPF exporter does not get deployed using `kubectl` and, therefore will not show up when you run “`kubectl get deployment -n monitoring`.”

4.2.4 Deployments for Evaluation

To generate metrics, logs, and traces for experiments, we must deploy some workload onto our cluster to monitor. We will use the OpenTelemetry Demo microservices since they come pre-instrumented for application-layer metrics and traces. Figure 4.9 shows the distributed nature of the microservices demo. These microservices can be deployed by running the helm scripts provided in Figure 4.8. By default, the Envoy proxy exports application-layer distributed trace information through port 4318. We can capture

these distributed trace IDs running “`kubectl port-forward svc/my-otel-demo-otelcol 4318:4318`”. Our Locust load generator should also automatically be deployed and accessible on `localhost:8080/loadgen/`, as shown in Figure 3.6. The Locust service defines the behavior of the users that will simulate the load on your network. Using Locust, we can define the users’ tasks, the number of users, and the rate at which new users will be spawned. We compare the effectiveness of distributed traces generated from application-layer libraries like OpenTelemetry against the proposed cloud-observability implementation to answer **RQ1** and **RQ2**.

```

1  #Install helm
2  curl -fsSL -o get_helm.sh \
3  https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
4  chmod 700 get_helm.sh
5  ./get_helm.sh
6
7  helm repo add open-telemetry \
8  https://open-telemetry.github.io/opentelemetry-helm-charts
9  helm install my-otel-demo open-telemetry/opentelemetry-demo

```

Figure 4.8. Deploying OpenTelemetry microservice demo

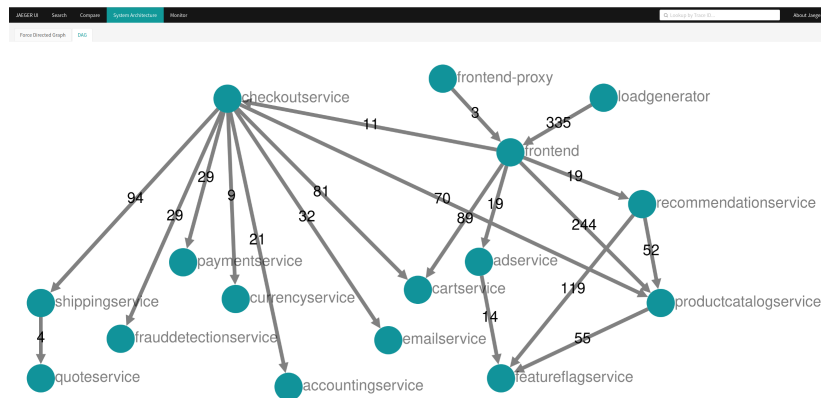


Figure 4.9. OpenTelemetry microservices demo architecture

4.2.5 Chaos Mesh

Finally, we install Chaos Mesh to trigger anomalies in our cluster and successfully answer **RQ3**. Like the OpenTelemetry demo, chaos mesh can be installed using helm charts and specifying Containerd as the runtime described in `cloud-observability/chaosmesh/install.txt`. We can utilize chaos mesh for various experiments in our Kubernetes cluster as described below.

1. **Network Chaos:** Network chaos experiments simulate various network failures and disruptions, including packet loss, latency, and bandwidth limits.
2. **Pod Chaos:** Pod chaos experiments simulate various pod-level failures, including killing pods, containers, or nodes.
3. **Time Chaos:** Time chaos experiments simulate time-related failures, such as clock skew or time delays.
4. **IO Chaos:** IO chaos experiments simulate various I/O-related failures, including disk or network I/O errors.
5. **Kernel Chaos:** Kernel chaos experiments simulate kernel-level failures, including memory exhaustion or CPU overload.
6. **DNS Chaos:** DNS chaos experiments simulate DNS-related failures, including DNS resolution failures or DNS timeouts.
7. **Stress Chaos:** Stress chaos experiments simulate various kinds of stress on the Kubernetes environment, including high CPU or memory usage.

4.3 Evaluation

We leveraged Chaos Mesh and Locust scripts to generate synthetic load on our application and test the effectiveness of our tool to monitor normal behavior. Cloud provides us with three essential utilities for our distributed applications, i.e., computing, storage, and networking. so we tested our solution’s ability to observe problems in any of these.

1. For generating baseline application metrics and traces, we used Locust to simulate users browsing and using our application.
2. For CPU and RAM load anomaly, we used the stress test feature of Chaos Mesh and deployed a resource-hungry container.
3. For I/O testing, we used the Chaos Mesh flexible I/O tester to perform random and sequential reads/writes with varying block sizes.
4. For network anomaly testing, we swarmed the Envoy sidecar proxy with TCP connections generated by Locust and identified the limitations of observability tools.

4.3.1 Performance Benchmarks

Once all our observability solutions were deployed, we used the `htop` command to analyze each tool’s computational overhead (CPU and RAM). These measurements were taken during average loads using Locust. Table 4.1 provides the exact values sampled and averaged over multiple measurements. As we can see, eBPF-based scripts consumed the lowest CPU overhead since the programs are simple and fast running. cAdvisor consumed the most resources since it needed to analyze and distinguish values between different cgroups of our Linux environment. It is important to note that the OpenTelemetry entry measures the resource needed from OTel Collector, a long-running daemon responsible for regularly polling OpenTelemetry instrumented microservices for metrics, logs and traces.

Table 4.1. CPU and RAM usage comparison

Tools	CPU	RAM
OpenTelemetry	2.8%	218 Mb
Envoy Sidecar	2.1%	110 Mb
NodeExporter	21.4%	140 Mb
cAdvisor	5.3%	75 Mb
eBPF Exporter	0.09%	84 Mb

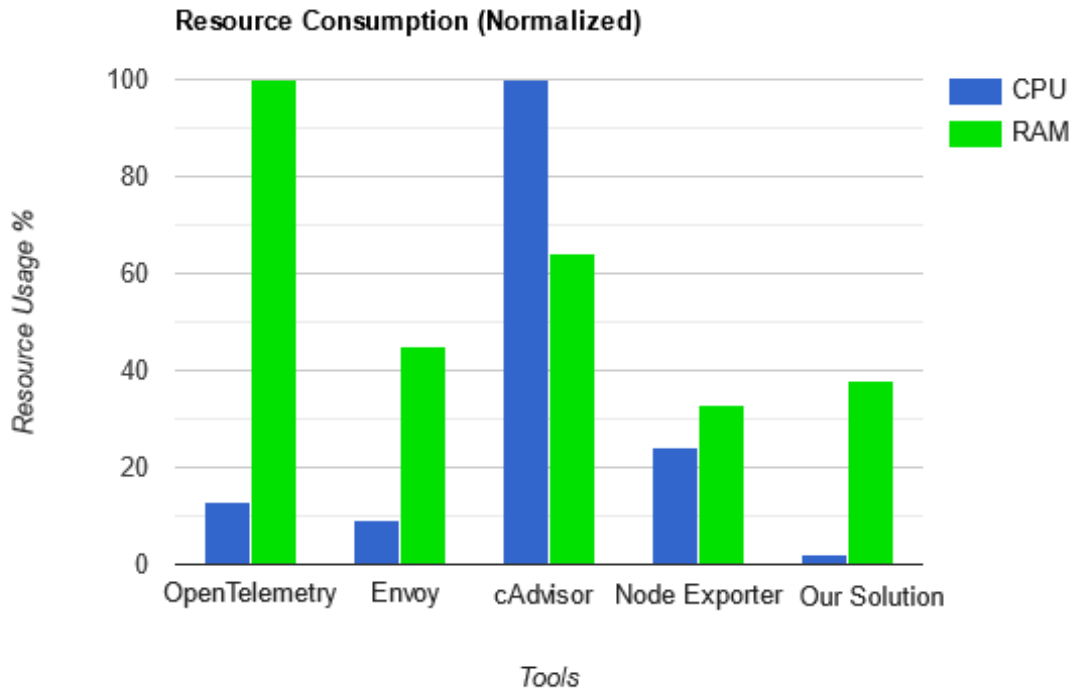


Figure 4.10. CPU and RAM usage comparison (normalized)

4.3.2 Synthetic Load Generation

We used a Locust script that simulated HTTP calls to our application for baseline synthetic load generation. Figure 4.11 can be referred to see the locust script used. The code snippet provided is our Locust load-testing script written in Python. It defines a class called `WebsiteUser` that extends the `HttpUser` class from Locust, representing a user interacting with a website over HTTP.

The `between` function is used to set the wait time between tasks for each simulated user. The wait time is randomly generated between the given minimum and maximum values. The `WebsiteUser` class defines several tasks that the simulated user can perform. Each task is decorated with the `@task` decorator, which assigns a weight to the task, indicating how fre-

quently it should be performed relative to the other tasks. For example, the `browse_product` task has a weight of 10, while the `checkout` and `checkout_multi` tasks have weights of 1.

To generate East-West traffic locust uses the `self.client` object to make HTTP requests to the microservice being tested. For example, the `browse_product` task makes a GET request to the product catalog microservice. At the `on_start` of the application we attach contextual baggage to all requests generated, these are part of the OpenTelemetry SDKs imported. Finally, the `on_start` function is called when the simulated user starts. In this case, it sets a synthetic request header using OpenTelemetry's baggage API and calls the `index` task to simulate the user visiting the Envoy front end proxy.

```

1  class WebsiteUser(HttpUser):
2      wait_time = between(1, 10)
3      @task(1)
4      def visit_homepage(self):
5          self.client.get("/")
6      @task(10)
7      def browse_random_product(self):
8          product = random.choice(products)
9          self.client.get(f"/api/products/{product}")
10     @task(3)
11     def get_product_recommendations(self):
12         params = {
13             "productIds": [random.choice(products)],
14         }
15         self.client.get("/api/recommendations/", params=params)
16     @task(3)
17     def view_ads(self):
18         params = {
19             "contextKeys": [random.choice(categories)],
20         }
21         self.client.get("/api/data/", params=params)
22     @task(1)
23     def complete_checkout(self):
24         # simulate a checkout process
25         user = str(uuid.uuid1())
26         self.add_product_to_cart(user=user)
27         checkout_person = random.choice(people)
28         checkout_person["userId"] = user
29         self.client.post("/api/checkout", json=checkout_person)
30     @task(1)
31     def checkout_multi(self):
32         user = str(uuid.uuid1())
33         num_items = random.choice([2, 3, 4])
34         for i in range(num_items):
35             self.add_product_to_cart(user=user)
36         checkout_person = random.choice(people)
37         checkout_person["userId"] = user
38         self.client.post("/api/checkout", json=checkout_person)
39     def on_start(self):
40         ctx = baggage.set_baggage("synthetic_request", "true")
41         context.attach(ctx)
42         self.index()

```

Figure 4.11. Locust script for synthetic load generation

4.3.3 Stress Tests

We performed stress tests in our Kubernetes cluster using the chaos mesh tool. Chaos mesh uses the Linux stress-ng library to run and perform floating point arithmetic in an infinite loop and increase CPU usage and malloc() calls to increase virtual memory usage. We instantly detected the stress test injected as described in Figure 4.12 on our Grafana Dashboard. If chaos mesh injects anomalies outside of container runtimes, it does not get picked up by cAdvisor but NodeExporter was able to identify the increased CPU and RAM load as shown in Figure 4.13. But, if we were to deploy a container that consumes high CPU and RAM like the xmrig cryptominer, cAdvisor could better identify the anomaly than NodeExporter, which only provided generalized information. In Figure 4.14, we can see that cAdvisor can pin-point which container and pod are malfunctioning. OpenTelemetry on the other hand was exporting CPU and RAM metrics that were able to detect the high stress instantly but achieving this required significant programming effort from the developers.

```
1  kind: StressChaos
2  apiVersion: chaos-mesh.org/v1alpha1
3  metadata:
4    namespace: default
5    name: stress
6  spec:
7    selector:
8      namespaces:
9        - default
10   mode: all
11   stressors:
12     memory:
13       workers: 10
14       size: 20GB
15     cpu:
16       workers: 30
17       load: 99
18   duration: 5m
```

Figure 4.12. CPU and RAM stress test YAML configuration

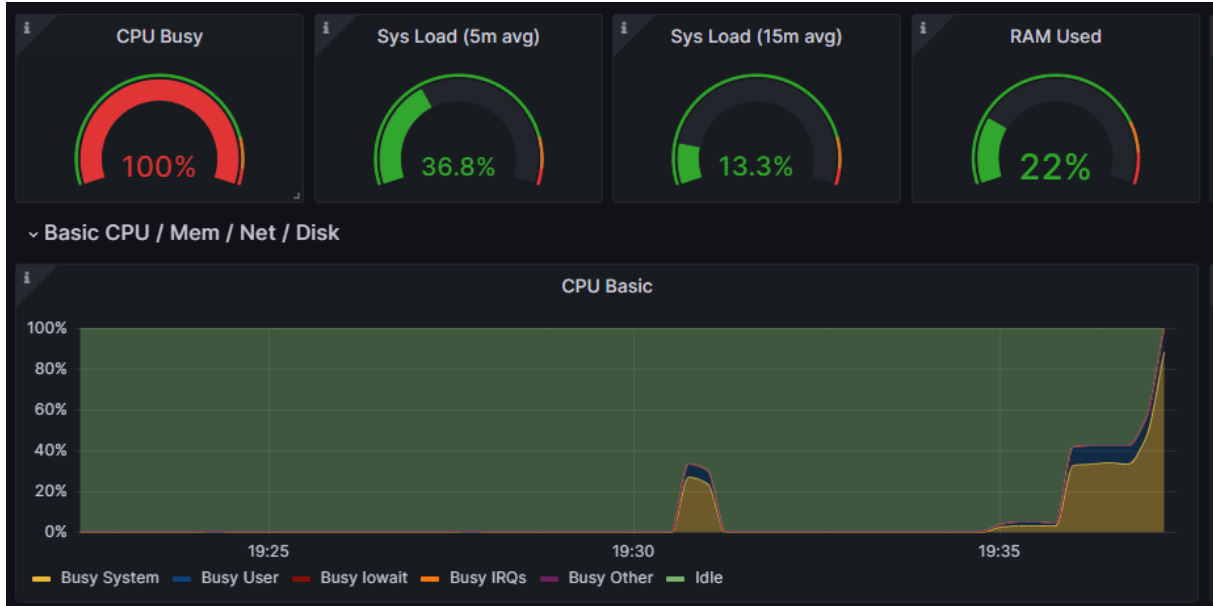


Figure 4.13. Node exporter identifying high CPU and RAM load during stress test

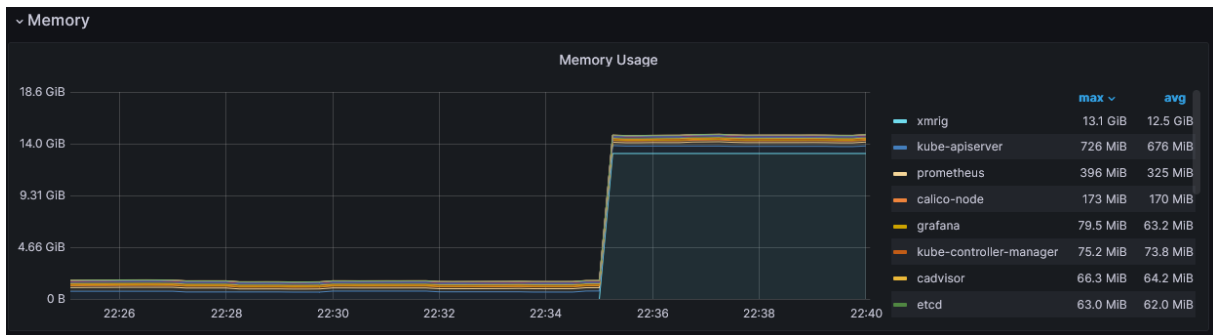


Figure 4.14. cAdvisor identifying high resource consumption pod in cluster

In Table 4.2, we detail which observability tools helped detect high CPU and RAM usage. Another key metric to observe while monitoring CPU and RAM is the number CPU cache misses. Cache misses occur when our CPU tries to reference a memory address that is no longer in low-level cache and needs to reference the RAM. Cache misses can slowdown the performance of our applications as reading from RAM is much slower than cache. Surprisingly we did not observe many cache misses during the stress test. This was

because the stress-ng libraries perform repeated arithmetic calculations during its CPU stress test; therefore, the cache has the required operations and data due to the reference locality.

Table 4.2. CPU and RAM observability

Tools	CPU Usage	RAM Usage	Cache Misses
OpenTelemetry (Application Layer)	✓	✓	X
Envoy Sidecar (Network Layer)	X	X	X
NodeExporter (Host Layer)	✓	✓	X
cAdvisor (Container Layer)	✓	✓	X
eBPF Exporter (Kernel Layer)	X	X	✓

4.3.4 I/O Performance

We performed I/O tests in our Kubernetes cluster using the chaos mesh tool. Chaos mesh uses the Linux Flexible IO Tester (Fio), which spawns several threads or processes doing a particular type of I/O action as specified by the user. We performed random reads/writes and sequential read writes on our worker nodes and checked whether our solution could detect this workload. Once again, we observed that NodeExporter and eBPF Exporter were the best indicators for I/O performance issues. Our block I/O latency eBPF exporter script was most helpful, which immediately showed us much longer read and write latencies in a histogram, something other tools cannot provide. Since eBPF scripts can store some amount of memory in the eBPF map, they can aggregate latency information locally and present it in a histogram format when scraped. Compared to Node Exporter samples data from the /proc folder and might lose information when displaying I/O metrics. Figure 4.15 is the visualization provided by NodeExporter, we can see that it can provide a timestamp of when

the I/O stress test began but it is still giving generalized information without providing a distribution or histogram. Figure 4.16 is the eBPF exporter dashboard generated by our `biolatency.bpf.c` script and visualized as a histogram on the Grafana dashboard. The eBPF exporter version of the visualization provides precise metrics about the duration of the I/O latency along with a distribution, but it cannot provide a timeline like NodeExporter.

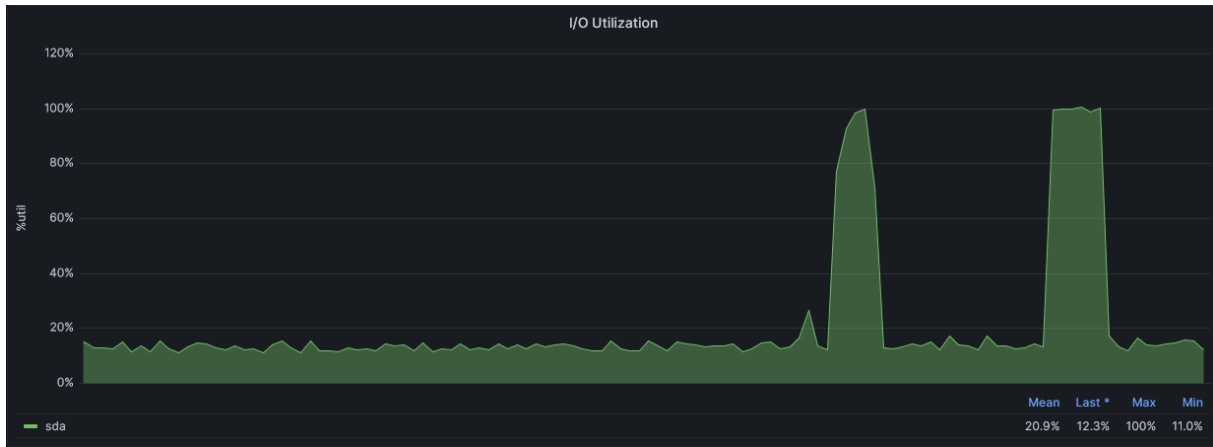


Figure 4.15. I/O stress test

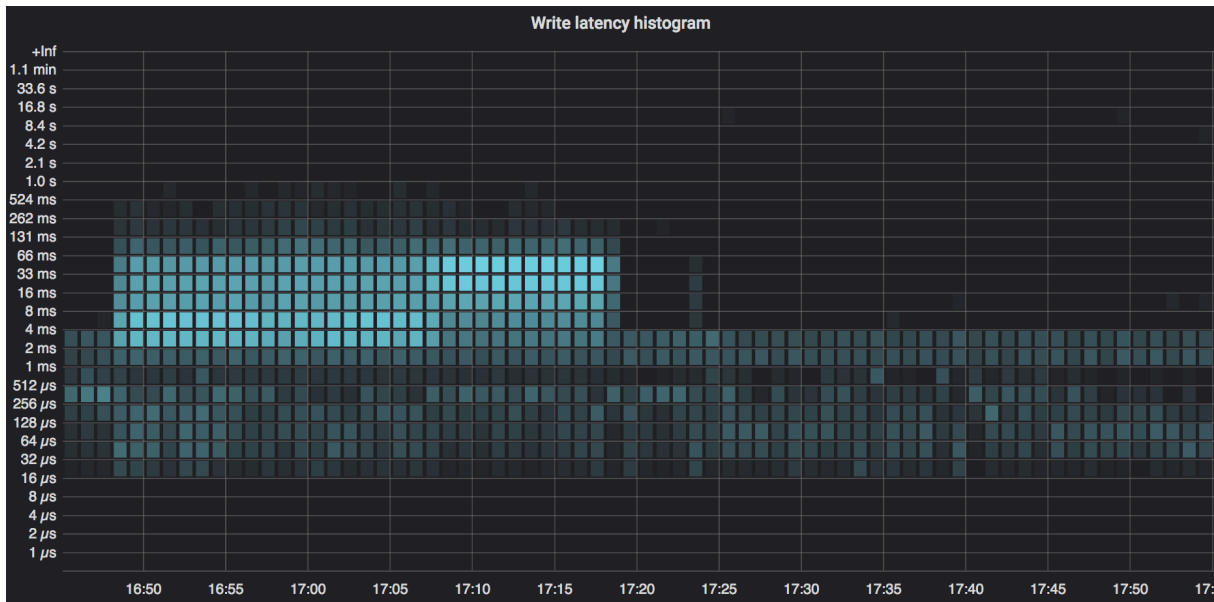


Figure 4.16. Block I/O latency histogram from eBPF exporter

4.3.5 Network Performance

The locust load generator described in Sec 4.3.2 directs requests to the Envoy frontend proxy exposed on port 8080. Envoy is responsible for adding distributed trace ID information to all requests, allowing us to track them in our network. OpenTelemetry collector can combine all the traces generated by microservices and the different endpoints accessed by the locust script. Once exported to Prometheus we can use Grafana for visualization of distributed traces, as shown in Figure 4.17.

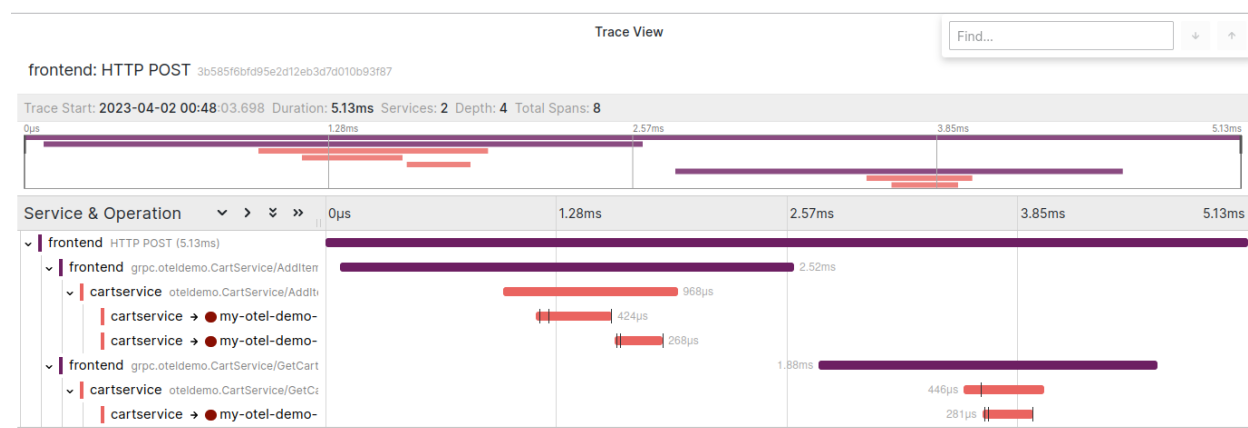


Figure 4.17. OpenTelemetry distributed trace visualized on Grafana dashboard

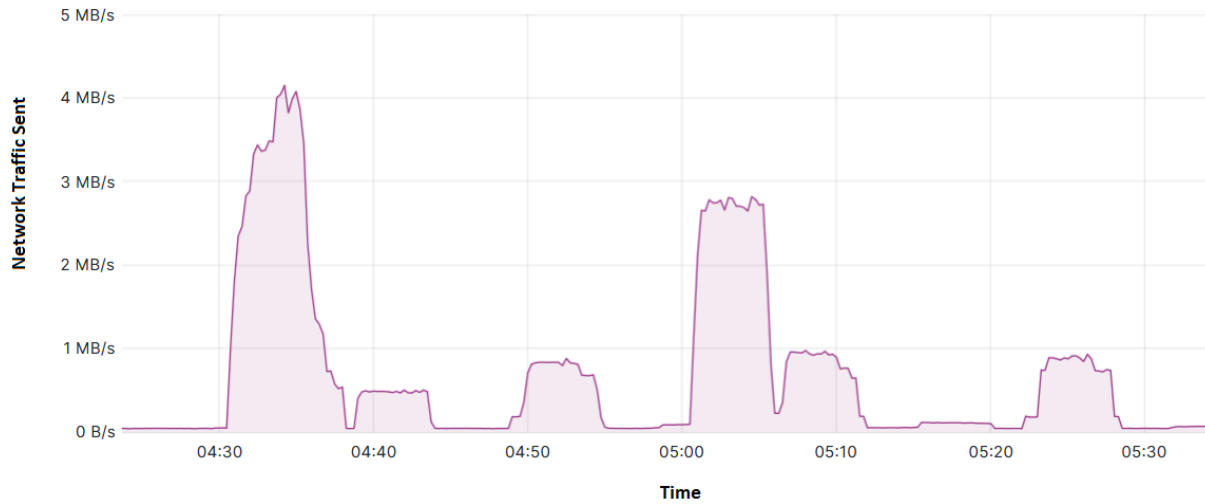
While distributed tracing is a powerful tool for monitoring and analyzing application performance, it is not enough for network observability. Network observability requires more comprehensive visibility into the network, including the performance and behavior of the network itself and the applications and services that rely on it. Some limitations of OpenTelemetry and Envoy-based network observability that we identified are:

1. **Limited visibility into the network:** Distributed tracing is only concerned with tracing the path of a request through the application stack, which may not provide enough visibility into the network itself. It is not be able to identify issues that are specific to the network, such as congestion or routing problems.
2. **Inability to capture low-level metrics:** Distributed tracing is able to capture high-level information about requests, such as their latency or the services they interact

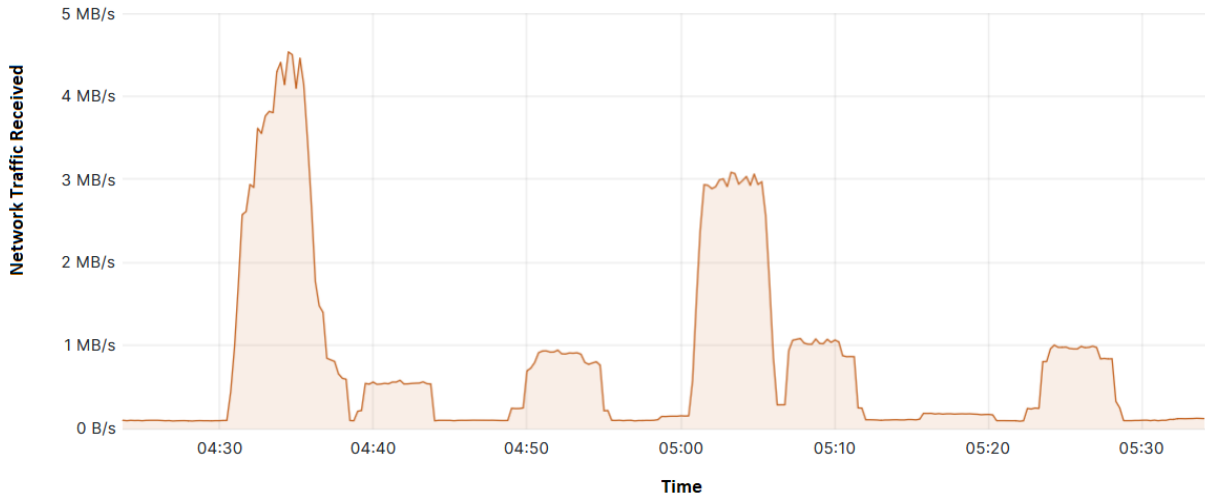
with. It does not capture lower-level details about network behavior, such as packet loss, latency variation, or network topology changes.

3. **Sampling:** Distributed tracing can generate a large volume of data that needs to be stored and processed, it is necessary to use sampling techniques which results in loss of holistic information.
4. **Operational overhead:** Distributed tracing requires instrumentation of all components in the application stack to generate trace data. We had the benefit of leveraging OpenTelemetry's demo which came pre-instrumented but it required considerable efforts from the open-source community to get distributed traces and not all microservices in the demo support them yet.

When running network load experiments, we observed that the distributed traces provided by Envoy could accurately identify network latencies in East-West network traffic but was not accurately representing all the requests. Since distributed traces perform sampling rather than simply recording all the requests it could not identify business critical metrics like total active users. Although we can access these metrics through the application-layer metrics exporter via OpenTelemetry, these are not always available in agile environments where developers cannot instrument the application layer. NodeExporter and cAdvisor identified the increase in network packets sent and packet received as shown in Figure 4.18. Using Node Exporter we can also get other Network information like MTU size, TCP connections, network speed, and traffic errors. Still, they could not distinguish whether the network traffic was East-West or North-South. Even cAdvisor which should provide container-level observability, failed to distinguish network traffic since cgroups and namespaces are not applicable for networking and cAdvisor only presents cumulative data for networking observability.



(a) Outgoing network traffic



(b) Incoming network traffic

Figure 4.18. cAdvisor and node exporter network monitoring

Our eBPF scripts `accept-latency.bpf.c` and `tcp-syn-backlog.bpf.c` did provide some insightful metrics regarding the networking performance of our Kubernetes nodes, but even they fail to provide insightful observability of the entire networking infrastructure. We hypothesize that to achieve network-layer observability for Kubernetes cluster truly, we must have a tightly integrated CNI that supports eBPF-based networking similar to Cilium. For example, if we used eBPF maps-based networking IP tables similar to the Cilium CNI,

we could keep track of networking data for each IP address and access it across pods to improve our network monitoring capabilities. This will be discussed in Section 5.2.

5. CONCLUSIONS AND FUTURE WORK

5.1 Conclusions

To solve the problem of microservice observability, this thesis aims at developing open-source toolkit that can collect and analyze application-layer, network-layer, and host-layer metrics for any Kubernetes deployment. We investigated the observability performance of NodeExporter, cAdvisor, eBPF exporter, Envoy and OpenTelemetry in a Kubernetes cluster. We also used the Chaos Mesh library to inject anomalies and detect them. The goal was to evaluate the effectiveness of each observability tool in detecting and mitigating anomalies in a distributed system.

Our experiments showed that OpenTelemetry and Envoy distributed traces were the most effective tools in detecting anomalies and providing a comprehensive view of the system. OpenTelemetry provided a holistic approach to observability by collecting metrics, traces, and logs, while Envoy distributed traces allowed for detailed tracing of service-to-service communication. However, OpenTelemetry and Envoy required the most rewriting of the application codebase to achieve this detailed observability. Another drawback of OpenTelemetry was the requirement of deploying an Opentelemetry Collector sidecar to ingest OLTP data. This added task of exporting OpenTelemetry metrics and traces to Prometheus. OpenTelemetry has attempted to enable push-mode data exporting directly to Prometheus but it is in not yet supported. This further increased the operational overhead of enable

NodeExporter and cAdvisor provided system-level metrics but were limited in providing application-level telemetry data. We also found that the eBPF exporter was a powerful tool for monitoring various metrics, including network traffic, system calls, and kernel events. eBPF exporter was highly flexible and could be integrated with other observability tools like Prometheus and Grafana.

While eBPF exporter has a steep learning curve and requires specialized knowledge of low-level system programming, it provides a more granular view of the system's performance than NodeExporter and cAdvisor. eBPF exporter can collect a wide range of telemetry data, making it useful for monitoring the performance of complex distributed systems. eBPF scripts do require programming effort similar to OpenTelemetry. Still, rather than

instrumenting each application independently, eBPF only requires one-time efforts by the developer to get observability for the entire distributed cluster.

Network-level observability was the most challenging to achieve. Our research highlights the importance of choosing the right observability tools for monitoring distributed systems in Kubernetes clusters. Depending on the needs of your application, you may choose one or more of these tools to provide observability and monitoring. OpenTelemetry and Envoy distributed traces provide a more comprehensive approach to monitoring the performance of complex distributed systems however, it requires extensive programming effort. NodeExporter and cAdvisor helped get system-level metrics without any programming required, but we could not extend their functionality to generate custom metrics. eBPF exporter provides a more detailed and flexible approach to monitoring system performance. The Chaos Mesh library was a valuable tool for evaluating the resilience of observability tools in detecting and mitigating anomalies in distributed systems.

5.2 Future Research Directions

This thesis tackles the problem of microservice observability in cloud-native environments in order to keep up with the fast-growing demand for cloud-based application deployments. There are still many exciting use cases of eBPF-based observability for further exploration. Here, we introduce three for potential future research directions: sidecarless service mesh, multi-cloud observability, and automated anomaly detection ML models.

Sidecarless service mesh. Service meshes have become the norm for managing networking between microservices-based applications in the cloud. None of the tools in our study were able to provide complete network-layer observability for Kubernetes-based workloads. Cilium, an open-source CNI for Kubernetes uses an eBPF-based data plane, which provides deep visibility into network traffic at the kernel level. eBPF functions allows for the creation of highly efficient and customized network probes that can capture detailed network traffic data in real-time. Cilium uses eBPF probes to collect detailed network layer metrics, including packet loss, latency, and throughput. Cilium also includes a distributed tracing fea-

ture that allows for the correlation of network-level observability data with application-level observability data, enabling end-to-end observability across the entire stack.

Multi-cloud observability. This thesis focuses on deploying our cloud observability solution on the chameleon cloud provider. Multi-cloud deployments are becoming increasingly popular for higher availability and vendor lock-in reduction. We did not explore the possibility of a truly homogeneous cluster configuration comprising nodes hosted on different clouds. Multi-cloud environments can be complex and diverse, with a variety of different operating systems, platforms, and cloud providers. eBPF provides a standardized way to instrument the Linux kernel across different environments, allowing for consistent monitoring and observability across multiple clouds.

Automated anomaly detection. AIOps, or Artificial Intelligence for IT Operations, is a methodology that uses AI and machine learning to improve the efficiency, speed, and accuracy of IT operations. This thesis provides a complete toolkit and data collection pipeline for observability data collection of the application-layer, network-layer and kernel-layer. The Prometheus data can be used for anomaly detection with machine learning by feeding the time-series data into machine learning models that can detect unusual patterns or trends that may indicate an issue with the IT infrastructure. Some algorithms that may be used for this work are isolation forests, autoencoder neural networks and one-class support vector machine. In order to use Prometheus data for anomaly detection with machine learning, the data needs to be preprocessed and transformed into a format that can be used by the machine learning algorithms. This may involve feature engineering to extract relevant features from the time-series data, normalization to ensure that the data is on a consistent scale, and splitting the data into training and testing sets for the machine learning models.

REFERENCES

- [1] S. Caculo, K. Lahiri, and S. Kalambur, “Characterizing the scale-up performance of microservices using teastore,” in *2020 IEEE International Symposium on Workload Characterization (IISWC)*, IEEE, 2020, pp. 48–59.
- [2] M. Villamizar, O. Garcés, H. Castro, M. Verano, L. Salamanca, R. Casallas, and S. Gil, “Evaluating the monolithic and the microservice architecture pattern to deploy web applications in the cloud,” in *2015 10th Computing Colombian Conference (10CCC)*, IEEE, 2015, pp. 583–590.
- [3] T. Ueda, T. Nakaike, and M. Ohara, “Workload characterization for microservices,” in *2016 IEEE International Symposium on Workload Characterization (IISWC)*, IEEE, 2016, pp. 1–10.
- [4] Y. Gan and C. Delimitrou, “The architectural implications of cloud microservices,” *IEEE Computer Architecture Letters*, vol. 17, no. 2, pp. 155–158, 2018.
- [5] Linkerd, *Linkerd: Ultralight, security-first service mesh for Kubernetes*, <https://linkerd.io/>, Accessed on March 28, 2023.
- [6] K. Morris, *Infrastructure as code: managing servers in the cloud*. " O’Reilly Media", 2016.
- [7] B. Cantrill, *Observability Practitioners Summit 2018*, <http://opsummitna18.sched.com/event/HfG2/visualizing-distributed-systems>, Accessed on March 28, 2023.
- [8] A. Dave, *Metrics, logs and traces: The Golden Triangle of Observability in monitoring*, <https://devops.com/metrics-logs-and-traces-the-golden-triangle-of-observability/>, Accessed on March 28, 2023.
- [9] P. Bourgon, *Metrics, tracing, and logging*, <https://peter.bourgon.org/blog/2017/02/2-1/metrics-tracing-and-logging.html>, Accessed on March 28, 2023.
- [10] L. A. Barroso, J. Dean, and U. Hölzle, “Web search for a planet: The google cluster architecture,” *IEEE Micro*, vol. 23, pp. 22–28, 2003.
- [11] S. S. Mike Loukides, *Microservices adoption in 2020*, <https://www.oreilly.com/radar/microservices-adoption-in-2020/>, Jul. 2020.

- [12] Kubernetes, *Kubernetes: Production Grade Container Orchestration*, <https://kubernetes.io/>, Accessed on March 28, 2023.
- [13] M. R. S. Sedghpour and P. Townend, “Service Mesh and eBPF-Powered Microservices: A Survey and Future Directions,” in *2022 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, IEEE, 2022, pp. 176–184.
- [14] eBPF, *Extended Berkeley Packet Filter*, <https://ebpf.foundation/>, Accessed on March 28, 2023.
- [15] OpenTelemetry, *The OpenTelemetry Project*, <https://www.opentelemetry.io/>, Accessed on March 28, 2023.
- [16] D. Nadig, B. Ramamurthy, B. Bockelman, and D. Swanson, “Optimized Service Chain Mapping and reduced flow processing with Application-Awareness,” in *2018 4th IEEE Conference on Network Softwarization and Workshops (NetSoft)*, 2018, pp. 303–307. DOI: 10.1109/NETSOFT.2018.8459912.
- [17] J. Levin and T. A. Benson, “ViperProbe: Rethinking microservice observability with eBPF,” in *2020 IEEE 9th International Conference on Cloud Networking (CloudNet)*, IEEE, 2020, pp. 1–8.
- [18] M. McCarter, *Nascio 2021: State CIO survey details Tech Priorities Post-pandemic*, <https://statetechmagazine.com/article/2021/10/nascio-2021-state-cio>, Accessed on March 28, 2023.
- [19] D. Nadig, B. Ramamurthy, and B. Bockelman, “SNAG: SDN-Managed Network Architecture for GridFTP Transfers Using Application-Awareness,” *IEEE/ACM Transactions on Networking*, vol. 30, no. 4, pp. 1585–1598, 2022. DOI: 10.1109/TNET.2022.3150000.
- [20] B. M. Abbott, “A security evaluation methodology for container images,” M.S. thesis, Brigham Young University, 2017.
- [21] K. Kaur, T. Dhand, N. Kumar, and S. Zeadally, “Container-as-a-service at the edge: Trade-off between energy efficiency and service availability at fog nano data centers,” *IEEE wireless communications*, vol. 24, no. 3, pp. 48–56, 2017.
- [22] Docker: *Container Platform*, <https://www.docker.com/>, Accessed on March 28, 2023.

- [23] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, omega, and kubernetes,” *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016.
- [24] D. Comer, “Microservices,” in *The cloud computing book: The future of computing explained*. CRC Press, Taylor and Francis Group, 2021.
- [25] A. Balalaie, A. Heydarnoori, and P. Jamshidi, “Microservices architecture enables devops: Migration to a cloud-native architecture,” *IEEE Software*, vol. 33, no. 3, pp. 42–52, 2016. DOI: 10.1109/MS.2016.64.
- [26] C. Sridharan, *Distributed Systems Observability: A Guide to Building Robust Systems*. O’Reilly Media, 2018.
- [27] Y. Shkuro, *Mastering Distributed Tracing: Analyzing performance in microservices and complex systems*. Packt Publishing Ltd, 2019.
- [28] B. H. Sigelman, L. A. Barroso, M. Burrows, P. Stephenson, M. Plakal, D. Beaver, S. Jaspán, and C. Shanbhag, “Dapper, a large-scale distributed systems tracing infrastructure,” Google, Inc., Tech. Rep., 2010. [Online]. Available: <https://research.google.com/archive/papers/dapper-2010-1.pdf>.
- [29] Zipkin, *Zipkin: A Distributed Tracing System*, <https://zipkin.io/>, Accessed on March 28, 2023.
- [30] Y. Shkuro, *Jaeger: Open Source, End-to-end Distributed Tracing*, <https://www.jaeger-tracing.io/>, Accessed on March 28, 2023.
- [31] Elastic, *Elastic Observability Guide*, <https://www.elastic.co/guide/en/observability>, Accessed on March 28, 2023.
- [32] Prometheus, *Prometheus: Monitoring System and Time Series Database*, <https://prometheus.io/>, Accessed on March 28, 2023.
- [33] Grafana, *Grafana: The open observability platform*, <https://grafana.com/>, Accessed on March 28, 2023.
- [34] W. Li, Y. Lemieux, J. Gao, Z. Zhao, and Y. Han, “Service mesh: Challenges, state of the art, and future research opportunities,” in *2019 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, IEEE, 2019, pp. 122–1225.

- [35] W. Li, Y. Lemieux, J. Gao, Z. Zhao, and Y. Han, “Service mesh: Challenges, state of the art, and future research opportunities,” in *2019 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, 2019, pp. 122–1225. DOI: 10.1109/SOSE.2019.00026.
- [36] O. Sheikh, S. Dikaleh, D. Mistry, D. Pape, and C. Felix, “Modernize digital applications with microservices management using the istio service mesh,” in *Proceedings of the 28th Annual International Conference on Computer Science and Software Engineering*, 2018, pp. 359–360.
- [37] NGINX, *What is Service Mesh*, <https://www.nginx.com/blog/what-is-a-service-mesh>, Accessed on March 28, 2023.
- [38] B. Hu, D. Nadig, and B. Ramamurthy, “Improving service performance through multilayer routing and service intelligence in a network service mesh,” in *2021 IEEE International Conference on Advanced Networks and Telecommunications Systems (ANTS)*, 2021, pp. 420–425. DOI: 10.1109/ANTS52808.2021.9936995.
- [39] D. Nadig, B. Ramamurthy, B. Bockelman, and D. Swanson, “April: An application-aware, predictive and intelligent load balancing solution for data-intensive science,” in *IEEE INFOCOM 2019 - IEEE Conference on Computer Communications*, 2019, pp. 1909–1917. DOI: 10.1109/INFOCOM.2019.8737537.
- [40] Envoy, *Envoy: High performance distributed proxy*, <https://www.envoyproxy.io/>, Accessed on March 28, 2023.
- [41] Grafana, *Observability survey report 2023*, <https://grafana.com/observability-survey-2023/>, Accessed on March 28, 2023.
- [42] J. Bastos and P. Araújo, *Hands-On Infrastructure Monitoring with Prometheus: Implement and scale queries, dashboards, and alerting across machines and containers*. Packt Publishing Ltd, 2019.
- [43] D. Uzunidis, P. Karkazis, C. Roussou, C. Patrikakis, and H. C. Leligou, “Intelligent performance prediction: The use case of a hadoop cluster,” *Electronics*, vol. 10, no. 21, p. 2690, 2021.

- [44] O. Mart, C. Negru, F. Pop, and A. Castiglione, “Observability in kubernetes cluster: Automatic anomalies detection using prometheus,” in *2020 IEEE 22nd International Conference on High Performance Computing and Communications; IEEE 18th International Conference on Smart City; IEEE 6th International Conference on Data Science and Systems (HPCC/SmartCity/DSS)*, 2020, pp. 565–570. DOI: 10.1109/HPCC-SmartCity-DSS50907.2020.00071.
- [45] Prometheus, *Monitoring linux host metrics with the node exporter: Prometheus*, <https://prometheus.io/docs/guides/node-exporter/>, Accessed on March 28, 2023.
- [46] E. Casalicchio and V. Perciballi, “Measuring docker performance: What a mess!!!” In *Proceedings of the 8th ACM/SPEC on International Conference on Performance Engineering Companion*, ser. ICPE ’17 Companion, L’Aquila, Italy: Association for Computing Machinery, 2017, pp. 11–16, ISBN: 9781450348997. DOI: 10.1145/3053600.3053605. [Online]. Available: <https://doi.org/10.1145/3053600.3053605>.
- [47] Prometheus, *Monitoring docker container metrics using cadvisor: Prometheus*, <https://prometheus.io/docs/guides/cadvisor/>, Accessed on March 28, 2023.
- [48] Facebook, *Katran: A high performance layer 4 load balancer*, <https://github.com/facebookincubator/katran>, Accessed on March 28, 2023.
- [49] Joelagnel, *Adeb: A debian-based shell environment designed for Android and adb*, <https://github.com/joelagnel/adeb>, Accessed on March 28, 2023.
- [50] H. M. Demoulin, I. Pedisich, N. Vasilakis, V. Liu, B. T. Loo, and L. T. X. Phan, “Detecting Asymmetric Application-layer Denial-of-Service Attacks In-Flight with FineLame,” in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, 2019, pp. 693–708.
- [51] L. Deri, S. Sabella, S. Mainardi, P. Degano, and R. Zunino, “Combining System Visibility and Security Using eBPF,” in *ITASEC*, 2019.
- [52] M. A. Vieira, M. S. Castanho, R. D. Pacfico, E. R. Santos, E. P. C. Júnior, and L. F. Vieira, “Fast packet processing with eBPF and XDP: Concepts, code, challenges, and applications,” *ACM Computing Surveys (CSUR)*, vol. 53, no. 1, pp. 1–36, 2020.
- [53] N. Amit and M. Wei, “The design and implementation of hyperupcalls,” in *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, 2018, pp. 97–112.

- [54] C. Cassagnes, L. Trestioreanu, C. Joly, and R. State, “The rise of eBPF for non-intrusive performance monitoring,” in *NOMS 2020-2020 IEEE/IFIP Network Operations and Management Symposium*, IEEE, 2020, pp. 1–7.
- [55] S. McCanne and V. Jacobson, “The BSD Packet Filter: A New Architecture for User-level Packet Capture,” in *USENIX winter*, vol. 46, 1993.
- [56] Isovalent, *Cilium: Linux-native, api-aware networking and security for containers*, <https://cilium.io/>, Accessed on March 28, 2023.
- [57] Iovisor, *Bpfttrace reference guide*, https://github.com/iovisor/bpfttrace/blob/master/docs/reference_guide.md, 2022.
- [58] IOvisor, *bpfttrace: High-level tracing language for Linux eBPF*, <https://tldp.org/LDP/Linux-Filesystem-Hierarchy/html/proc.html>, Accessed on March 28, 2023.
- [59] M. Pawlikowski, *Chaos engineering*. O’Reilly Media, 2021.
- [60] C. Mesh, *Chaos Mesh: A powerful Chaos Engineering Platform for Kubernetes*, <https://chaos-mesh.org/>.
- [61] C. Liu, Z. Cai, B. Wang, Z. Tang, and J. Liu, “A protocol-independent container network observability analysis system based on eBPF,” in *2020 IEEE 26th International Conference on Parallel and Distributed Systems (ICPADS)*, IEEE, 2020, pp. 697–702.
- [62] J. G. Almaraz-Rivera, “An anomaly-based detection system for monitoring kubernetes infrastructures,” *IEEE Latin America Transactions*, vol. 21, no. 3, pp. 457–465, 2023.
- [63] M. Song, Q. Liu, and H. E., “A mirco-service tracing system based on istio and kubernetes,” in *2019 IEEE 10th International Conference on Software Engineering and Service Science (ICSESS)*, 2019, pp. 613–616. DOI: 10.1109/ICSESS47205.2019.9040783.
- [64] Thrivikraman, V. R. Dixit, N. R. S, V. K. Gowda, S. K. Vasudevan, and S. Kalambur, *Misertrace: Kernel-level request tracing for microservice visibility*. DOI: 10.48550/ARXIV.2203.14076. [Online]. Available: <https://arxiv.org/abs/2203.14076>.

- [65] E. Schwartz, “A payment protocol of the web, for the web: Or, finally enabling web micropayments with the interledger protocol,” in *Proceedings of the 25th International Conference Companion on World Wide Web*, ser. WWW ’16 Companion, Montréal, Québec, Canada: International World Wide Web Conferences Steering Committee, 2016, pp. 279–280, ISBN: 9781450341448. DOI: 10.1145/2872518.2889305. [Online]. Available: <https://doi.org/10.1145/2872518.2889305>.
- [66] Y. Gan, Y. Zhang, D. Cheng, A. Shetty, P. Rathi, N. Katarki, A. Bruno, J. Hu, B. Ritchken, B. Jackson, K. Hu, M. Pancholi, Y. He, B. Clancy, C. Colen, F. Wen, C. Leung, S. Wang, L. Zaruvinsky, M. Espinosa, R. Lin, Z. Liu, J. Padilla, and C. Delimitrou, “An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud and Edge Systems,” in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS ’19, Providence, RI, USA: Association for Computing Machinery, 2019, pp. 3–18, ISBN: 9781450362405. DOI: 10.1145/3297858.3304013. [Online]. Available: <https://doi.org/10.1145/3297858.3304013>.
- [67] J. Salazar and N. R. Ivanko, *Security Observability with eBPF*. O’Reilly Media, 2022.
- [68] D. Calavra and L. Fontana, *Linux Observability with BPF*. O’Reilly Media, 2022.
- [69] M. Schiesser, *What is an api gateway*, <https://glasnostic.com/blog/what-is-an-api-gateway-aws-express-kong/index.html>, Accessed on March 28, 2023.
- [70] The Linux Documentation Project, *The /proc Filesystem*, <https://github.com/iovisor/bpfttrace>, Accessed on March 28, 2023.
- [71] L. Jackson, *Debugging microservices: Lessons from google, facebook, lyft*, <https://thenewstack.io/debugging-microservices-lessons-from-google-facebook-lyft/>, Accessed on March 28, 2023.
- [72] Ivan Babrou, *Introducing eBPF Exporter*, https://blog.cloudflare.com/introducing-ebpf_exporter/, Accessed on March 28, 2023.
- [73] P. Luszczek, E. Meek, S. Moore, D. Terpstra, V. M. Weaver, and J. Dongarra, “Evaluation of the hpc challenge benchmarks in virtualized environments,” in *European Conference on Parallel Processing*, Springer, 2011, pp. 436–445.

- [74] K. Keahey, J. Anderson, Z. Zhen, P. Riteau, P. Ruth, D. Stanzione, M. Cevik, J. Colleran, H. S. Gunawi, C. Hammock, J. Mambretti, A. Barnes, F. Halbach, A. Rocha, and J. Stubbs, “Lessons learned from the chameleon testbed,” in *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*, USENIX Association, Jul. 2020.